Journal of Information Security and Applications 92 (2025) 104096

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

A systematic literature review of log-correlation tools for cyberattack detection and prediction in large networks

Edward Chuah [a,*], Harsha Kalutarage [b], Kasim Tasdemir [c], Atnafu Abrham [d], Carsten Maple [e]

[a] University of Aberdeen, Aberdeen AB24 3FX, UK
[b] Robert Gordon University, Aberdeen, AB10 7AQ, UK
[c] Queen's University of Belfast, Belfast, BT7 1NN, UK
[d] ECDSWCo, Addis Ababa, Ethiopia
[e] University of Warwick, Coventry, CV4 7AL, UK

ARTICLE INFO

Keywords:
Systematic literature review (SLR)
Quality model
Log-correlation tools (SysLogCTs)
Cyberattack detection
Cyberattack prediction
NetFlow data
System logs
Cybersecurity data analytics

ABSTRACT

System logs are the main source of information available to security analysts to detect and predict cyberattacks on their networks. For example, enterprise networks generate a large volume of system logs including NetFlow data, Linux and Microsoft Windows host logs from multiple routers, servers and client devices. System log-analysis tools that detect or predict cyberattacks in a large network typically analyze the NetFlow data. Therefore, the idea of using a single data source to achieve a given objective, such as detecting or predicting a cyberattack, is losing its validity. Several system log-analysis tools have been developed to assist the security analyst in gaining an understanding of the behavior of their networks. These system log-analysis tools enable security analysts to perform various analyses that include detecting or predicting a cyberattack. The current system log-analysis tools vary significantly in their design and function. A Log-Correlation tool (SysLogCT) is a tool that uses a correlation method in its workflow to extract features or identify patterns of events in the data. We conduct a systematic review of literature (SLR) on SysLogCTs for detecting and predicting cyberattacks, and select 62 representative articles out of 4,599 initial articles. Our selection criteria consist of: (a) general criteria, (b) inclusion criteria, (c) exclusion criteria, and (d) quality assessment criteria. Then, we propose a quality model to evaluate those SysLogCTs. To the best of our knowledge, there is no work that studied the characteristics of SysLogCTs which detect or predict cyberattacks with respect to six quality attributes consisting of: (a) spurious relationships, (b) missing values in the data, (c) correlation threshold value, (d) bias and variance, (e) anomalies in the data, and (f) evaluation metrics. Through our SLR, we (a) identify articles on SysLogCTs which detect or predict cyberattacks, (b) build our quality model consisting of the six quality attributes, (c) use our quality model to evaluate and recommend SysLogCTs, and (d) discuss several challenges for future research. Our work highlights the advantages and limitations of existing SysLogCTs that detected or predicted cyberattacks, and identifies research opportunities that could facilitate better detection and prediction of cyberattacks in large networks.

1. Introduction

Large organizations operate multiple types of networks. These networks generate a large amount of system logs which contain valuable information for detecting and predicting cyberattacks. In the context of cyberattack detection and prediction, we define a System Log-Correlation Tool (SysLogCT) as an approach that uses log-correlation to detect or predict a cyberattack in a large network. Log-Correlation is a process that uses or implements a correlation method to identify correlated messages in system logs. A background on the log-correlation process is provided in Section 4. We selected the relevant and representative SysLogCT approaches based on four selection criteria consisting of general criteria, inclusion criteria, exclusion criteria, and quality assessment criteria. These criteria are described in detail in Section 3.3. While commercial and open-source off-the-shelf Security Information and Event Management (SIEM) systems provide real-time monitoring of

* Corresponding author.
E-mail addresses: thuan.chuah@abdn.ac.uk (E. Chuah), h.kalutarage@rgu.ac.uk (H. Kalutarage), kasimtasdemir@gmail.com (K. Tasdemir), atnafu.abraham@dec.edu.et (A. Abrham), cm@warwick.ac.uk (C. Maple).

https://doi.org/10.1016/j.jisa.2025.104096

Available online 24 May 2025
2214-2126/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

network traffic, development of workflow pipelines, interactive dashboards and support for forensic analysis, SIEM systems can benefit from the advanced analytics that a SysLogCT approach provides [1,2]. Large networks generate millions of system log-messages in one day. These log-messages are captured in different types of system logs including, but not limited to network flow logs, authentication logs, software process logs, Domain Name Server (DNS) logs, and system logs that contain execution traces of malware detected by an antivirus software. Consequently, security analysts need SysLogCT approaches that integrate the heterogeneous data and provide advanced analytics to support detection and prediction of cyberattacks.

The system logs contain a wealth of valuable information that can help the security analyst understand the behavior of large networks. Here, the term "NetFlow" is used in a generic way to describe a standard logging protocol used by large networks to record and summarize communications between devices on the network [3]. NetFlow was conceptualized over three decades ago as an Internet Protocol (IP) monitoring system that allowed network users and applications to analyze the NetFlow data. Using the NetFlow protocol, standardized logging allowed the security analyst to obtain an overall picture of the volume of traffic flows on the network. The NetFlow data contain log-events that summarize the amount of traffic exchanged between two devices on a network and the duration of communication between the two network devices. While NetFlow is currently used by several large networks, more recently, the changes to usage patterns and data volume are highlighting its limitations, due to attacks that occur at the network routers and hosts. For example, enterprise networks allow its users to login and execute complex tasks on the servers, and data access and processing have become the main consumer of resources on these networks. These networks generate a large volume of heterogeneous data, which makes it difficult to integrate the different types of data for analysis [4].

To address this issue, many log-analysis tools have been developed to integrate and analyze the data. These log-analysis tools use multiple statistical techniques, data mining algorithms and/or machine learning models. These tools are also designed to achieve a different goal, for example alert analysis [5], intrusion detection [6], detecting network traffic anomalies [7] or predicting network traffic [8]. Several SysLogCT approaches are presented in the literature and security analysts have a wide range of SysLogCTs to choose from. However, they need to choose a SysLogCT approach that contains quality attributes which are best suited for their use case. For example, in cyberattack detection, a SysLogCT approach must be robust against anomalies in the data, missing data values and spurious correlations. A SysLogCT approach must avoid generating alerts which are correlated but not causally related. Several SysLogCT approaches use the correlation threshold value to identify strongly correlated alerts. Therefore, a SysLogCT approach must allow the correlation threshold settings to be tuned so that a cyberattack can be detected or predicted accurately [9]. However, there is a gap between the needs of security analysts and researchers. Security analysts are not aware of the quality attributes of SysLogCT approaches, They do not have the time to search through the large body of literature to identify a SysLogCT approach that can meet their requirements. Moreover, they do not have the time to review the characteristics of each SysLogCT approach. In contrast to security analysts, researchers focus on developing new system log-analysis techniques. To bridge this gap, our paper makes the following contributions:

- We conduct a systematic literature review of relevant and representative SysLogCT approaches that detected or predicted cyberattacks in a large network.
- We identify the quality attributes of SysLogCT approaches through our SLR, use these quality attributes to build a quality model, and apply our quality model to compare several SysLogCT approaches. Following on our search for initial articles that developed correlation-based approaches using system logs, we focus our comparison on SysLogCT approaches that analyze the network events because SysLogCT approaches which analyze correlations between network events and other types of events are not common.
- We recommend SysLogCT approaches for several use cases and discuss the advantages and limitations of these SysLogCT approaches.
- We present directions for future research in SysLogCT approaches for detecting and predicting cyberattacks.

We learned that there is not one SysLogCT approach that met all the six quality attributes. Researchers have focused on developing new statistical approaches or machine learning models to improve detection or prediction accuracy, but few works have considered spurious relationships, and no works have considered missing data values. Spurious relationships and missing values in the data can cause a SysLogCT approach to produce an incorrect detection or prediction of a cyberattack. Thus, the ability of a SysLogCT approach to identify spurious relationships and missing data values needs to be investigated to reduce the risk of making a wrong decision. Outlier or anomalous values in the data can affect the stability of a correlation algorithm or regression model when detecting or predicting a cyberattack. Therefore, it is important to develop correlation approaches that are robust against outliers in the data. The correlation threshold value is typically used in a correlation algorithm to identify a smaller set of network events for analysis, but determining the optimal correlation threshold value is a manual and time-consuming task. The bias and variance of a SysLogCT approach affect its ability to detect or predict an actual cyberattack accurately. The evaluation metrics that are used to evaluate the performance of a SysLogCT approach in detecting or predicting a cyberattack can increase the risk of missing an attack on the network, which reduces the security analyst's ability to respond to the attack. Our results are based on a thorough review of current SysLogCT approaches that detected or predicted cyberattacks. A summary of the main functions in the 26 unique SysLogCT approaches is given in Table 1.

The remainder of this paper is structured as follows. A review of related works is presented in Section 2. The design of our SLR is described in Section 3, followed by a detailed description on the log-correlation process in Section 4. A summary of the approaches identified through our SLR and described by the 26 SysLogCT approaches is presented in Section 5. Our quality model is described in Section 6, and the evaluation results on the 26 SysLogCT approaches are presented in Section 7. The limitations of our SLR is discussed in Section 8, and a summary is provided in Section 9.

2. Related work

In this section, we review the recent survey papers according to four related topics: (a) security event correlation, (b) cyberattack prediction, (c) intrusion detection and prevention, and (d) system log-analysis.

Kotenko et al. [36] presented a detailed and comprehensive review of papers that developed correlation methods and architectural solutions for security. They applied a systematic literature review approach and classified the reviewed papers into eight categories that included the year of publication, type of detection, sources of information, correlation methods, knowledge extraction methods, datasets used, architectural solutions and metrics used to evaluate the security event correlation approaches. They proposed six promising research areas that included analyzing the semantics of security events, developing and using self-learning knowledge bases, applying correlation methods to perform predictive analysis of security events, processing big data, unifying security events and developing new datasets. Then, Levshun and Kotenko [37] presented a systematic review of literature that developed rule, semantics, graphical and machine learning models that correlated security events. They proposed six research challenges that need to be addressed. The six challenges they proposed included detecting multi-step attacks, understanding the semantics of security events

Table 1
Main functions of SysLogCT approaches.

Tool	Goal	Data preprocessing	Multiple datasets	Attack type
Xiao *et. al.* [10]	Detection	Yes	Yes	Distributed Denial-of-Service (DDoS)
Ramaki et al. [11]	Detection & prediction	Yes	No	Generic
Gottwalt et al. [12]	Detection	Yes	No	Generic
Prasad et al. [13]	Detection	Yes	No	Generic
Mahdavi et al. [14]	Detection	Yes	Yes	Generic
Lajevardi et al. [15]	Detection	Yes	No	Advanced Persistent Threat (APT)
Zhang et al. [16]	Detection	Yes	Yes	APT
Wei et al. [17]	Detection	Yes	No	Reflection DDoS
Vidal et al. [18]	Detection	Yes	No	Generic
Khosravi et al. [19]	Detection	Yes	No	APT
Kidmose et al. [20]	Detection	Yes	No	Malware
Zhang et al. [21]	Detection	Yes	No	Worm
Mishra et al. [22]	Detection	Yes	No	DDoS
Padmashree & Krishnamoorthi [23]	Detection	Yes	No	Generic
Ghafir et al. [24]	Detection & prediction	Yes	No	APT
Cheng et al. [25]	Detection	Yes	No	Generic
Chuah et al. [26]	Identification	Yes	No	Reflection DDoS
Haas et al. [27]	Identification	Yes	Yes	Generic
Chuah & Suri [28]	Identification	Yes	No	Reflection DDoS
Amma & Subramanian [29]	Detection	Yes	Yes	Generic
Kidmose et al. [30]	Detection	Yes	Yes	Malware
Revathi [31]	Detection	Yes	No	Reflection DDoS DDoS
Chawla et al. [32]	Discrimination	Yes	Yes	DDoS & flash events
Diakhame et al. [33]	Reconstruction	Yes	No	Generic
Yang et al. [34]	Detection	Yes	No	APT
Grammatikis et al. [35]	Detection	Yes	No	Generic

detected, efficiency of deep learning models in processing multiple categorical event features, analyzing large volume of heterogeneous data, creating security event knowledge bases, and availability of real-world cybersecurity datasets.

Velásquez et al. [38] presented a systematic review of literature focused on the near and long-term future of technologies implemented in current SIEM systems. They highlighted the use and compatibility of SIEM systems with state-of-the-art technologies that included Blockchain, cloud infrastructures, containers and compliance with international privacy laws. The authors proposed a new framework called *SIEM-SC* that uses Blockchain, encryption and containers. They showed that *SIEM-SC* complied with the requirements of the General Data Protection Regulation Act, provided strong protection against unauthorized access and brute-force access, and handled large number of encryption keys efficiently.

de Neira et al. [39] presented an extensive review of literature on the state-of-the-art DDoS prediction techniques. They categorized these DDoS prediction techniques according to the architecture, methodology employed, time to predict a DDoS attack and datasets used. The authors discussed several open challenges for future research that include using reinforcement learning to predict DDoS attacks, using statistical models to increase the time for predicting a DDoS attack, and implementing hybrid solutions to enhance prediction of DDoS attacks.

Khraisat et al. [40] presented a comprehensive review of literature on recent intrusion detection systems, classified these intrusion detection systems according to the method of attack, and evaluated the available intrusion detection datasets. They discussed several open research challenges that included protecting intrusion detection systems from adversarial attacks and enhancing intrusion detection systems in detecting evasion attacks. Chou et al. [41] presented a broad view of literature on data-driven intrusion detection systems. They identified the methods developed in the surveyed data-driven intrusion detection systems, and categorized these data-driven intrusion detection systems based on the method developed. The authors presented a taxonomy of open research challenges that include the lack of real-world network datasets, handling redundant data, big data and small data, and robustness of the machine learning model from adversarial attacks. Gupta et al. [42] provided an up-to-date review of literature on intrusion detection systems, intrusion prevention systems and intrusion detection and response systems. They described the advantages and disadvantages of these systems and highlighted several areas for future research. The highlighted future research included the need for real-world cybersecurity datasets, reducing false alarms, increasing detection response time and reducing the computing resources consumed by these systems.

El-Masri et al. [43] performed a systematic review of literature on automated log-abstraction techniques, built a quality model that comprised of seven attributes and applied their quality model to evaluate existing automated log-abstraction techniques. They presented several promising research directions that include dynamically tuning the model parameters, sharing industrial log-files with researchers, reducing the reliance on domain knowledge to update rules in system log-abstraction tools, scaling these tools to abstract large log-files efficiently, and investigating novel approaches such as distributed architectures to enhance the efficiency of these techniques. Chuah et al. [44] conducted a systematic review of literature on system log-analysis tools that diagnosed or predicted failures in cluster systems. They developed a quality model that comprised of four attributes and compared 14 unique system log-correlation tools using their quality model. The authors discussed four research challenges and made a number of recommendations for future research that included reducing spurious correlations, tuning parameters in correlation models, increasing the robustness of correlation models against outliers in the data, and handling missing log-events.

Summary: To the best of our knowledge, there is no work that evaluated SysLogCT approaches that detected or predicted cyberattacks in a large network using six quality attributes which are identified through a systematic literature review. We have summarized the focus of the reviewed survey papers in Table 2. Differently to the articles in Table 2, we: (a) conduct a systematic review of SysLogCT approaches that detected or predicted cyberattacks in large networks, (b) develop a quality model consisting of the six quality attributes of spurious relationships, missing data values, outliers in the data, correlation threshold value, bias and variance, and evaluation metrics, and (c) apply our quality model to evaluate these SysLogCT approaches.

Table 2
Summary of related survey works.

Paper	Topic	System	Quality model	Quality attributes	Goal
Kotenko et al. [36]	Security event correlation	Generic	No	No	Multi-step & advanced persistent threat detection
Levshun and Kotenko et al. [37]	AI-based correlation	Generic	No	No	Security monitoring
Velásquez et al. [38]	SIEMs	Generic	No	No	Security compliance
de Neira et al. [39]	DDoS prediction	Computer networks	No	No	DDoS attack mitigation
Khraisat et al. [40]	IDS	Generic	No	No	Attack mitigation
Chou et al. [41]	Data-driven network IDS	Computer networks	No	No	Novel threats detection
Gupta et al. [42]	Intrusion prevention & detection (IPDS)	Generic	No	No	IPDS development
El-Masri et al. [43]	Automated log-abstraction	Generic	Yes	Mode, coverage, efficiency, scalability, delimiter independence, system knowledge independence, parameter tuning effort.	Software debugging
Chuah et al. [44]	System log-correlation	Cluster systems	Yes	Spurious correlation, Missing data, Data outliers, Correlation threshold setting.	Failure diagnosis & prediction
This paper	**System log-correlation**	**Large networks**	**Yes**	**Spurious correlation, Missing data, Data outliers, Correlation threshold settings, Bias & variance, Evaluation metrics.**	**Cyberattack detection & prediction**

3. Study design

Our SLR is conducted using guidelines from the Evidence-Based Software Engineering (EBSE) method given by Kitchenham et al. [45–47] for conducting a systematic literature review in software engineering. Their recommended approach for conducting an SLR is composed of five activities: (a) framing the research question, (b) identifying the relevant works, (c) assessing the quality of those works, (d) summarizing the evidence in those studies, and (e) interpreting the findings. We follow their SLR approach and use it to define our research question, select the relevant papers on SysLogCT approaches, compare the quality of the SysLogCT approaches, collate the papers on SysLogCT approaches and discuss our results. We use Google Scholar to obtain an initial set of articles. The reason for using Google Scholar is that it provides access to articles that are indexed in digital libraries such as IEEE, ACM, Elsevier and Springer. We obtained a set of 4,599 papers and identified a set of 39 seed papers. Then, we use Scopus to perform forward snowballing and obtained all papers that cited the seed papers. We perform backward snowballing by obtaining all references contained in those seed papers. The general criteria, inclusion criteria, exclusion criteria, and quality assessment criteria are given in Section 3.3. We obtained 62 representative papers, from which we identified 26 unique SysLogCT approaches that detected or predicted cyberattacks in a large network.

3.1. Research questions

First, we define the research questions and propose a quality model based on the answers to these research questions. Then, we use our quality model to evaluate the current SysLogCT approaches. Our research questions are as follows:

- What are the relevant and representative SysLogCT approaches for detecting cyberattacks, predicting cyberattacks or detecting and predicting cyberattacks?
- What are the quality attributes contained in those SysLogCT approaches?

3.2. Search approach

We studied articles that are published in conferences and journals. These articles were published between October 2010 and July 2024, and all articles are written in English. We decided to start our search from 2010 because Vaarandi and Podins [48] developed one of the first open-source network alert monitoring and analysis tool. We focus our search for initial articles that applied a correlation-based approach on system logs to detect or predict cyberattacks. Therefore, we use the following query strings on Google Scholar: (a) "`log analysis`", (b) "`log correlation`", (c) "`alert correlation`", (d) "`attack detection`", and (e) "`attack prediction`".

3.3. Selection approach

We developed a process that consist of three stages and use this process to screen for articles. The process is composed of: (a) general criteria, (b) inclusion and exclusion criteria, and (c) quality of the article. Our *general criteria* are given in Table 3.

We applied an equal weight of 100% to all criteria listed in Table 4, Tables 5 and 6. Our *inclusion criteria* are given in Table 4. If an article satisfies all the two inclusion criterion listed in Table 4, then the article is included.

Our *exclusion criteria* are given in Table 5. If an article satisfies at least one of the exclusion criterion listed in Table 5, then the article is excluded.

Our *quality assessment criteria* are given in Table 6. If an article satisfies all the four quality assessment criterion listed in Table 6, then the article is included.

Fig. 1 depicts our search and selection workflow which is composed of three phases: (a) seed articles identification, (b) candidate articles identification, and (c) final article set identification. Next, we describe each of the three phases in our search and selection workflow.

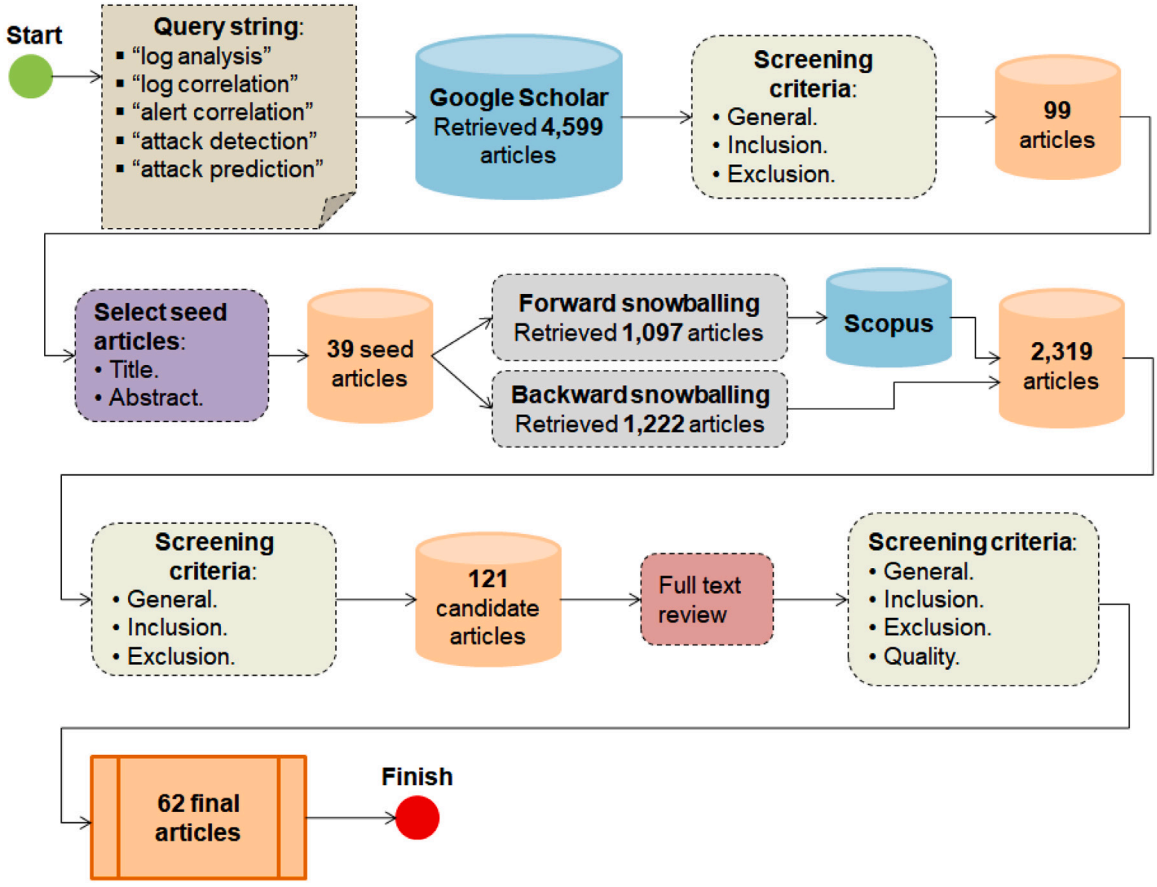


Fig. 1. Our search and selection workflow. It consists of: (a) identifying seed articles, (b) identifying candidate articles, and (c) identifying the final set of articles.

Table 3
General criteria.

No.	Criterion
G1	The paper is published between January 2010 and July 2024.
G2	The paper is published in a peer-reviewed journal or a peer-reviewed conference.
G3	The paper pertains to system log-analysis for large systems including enterprise networks, cloud computing systems and IoT networks. These systems generate large amounts of heterogeneous data and log-analysis tools are used to detect cyberattacks, predict cyberattacks or detect and predict cyberattacks on those systems. Therefore, we focus on system log-analysis papers for enterprise networks, cloud computing systems and IoT networks.

Table 4
Inclusion criteria.

No.	Criterion
I1	The paper is written in English
I2	The paper presented, proposed, developed or implemented a system log-correlation approach or a system log-correlation method

Table 5
Exclusion criteria.

No.	Criterion
E1	When a similar programming script, statistical approach, data mining algorithm or machine learning technique is proposed, presented, developed or implemented by two or more papers, at least one older paper is selected and excluded from the final set of papers.
E2	Papers that are not peer-reviewed.
E3	Papers describing how to generate, write or integrate system logs.
E4	Papers where access to the source code are required to evaluate the method.
E5	Papers presenting system logging workflows or system logging architectures.

Table 6
Quality assessment criteria.

No.	Criterion
Q1	The paper describes the research problem clearly.
Q2	The paper describes the proposed method clearly.
Q3	The paper describes all steps for evaluating the proposed method clearly.
Q4	The method proposed in the paper addressed all the research questions set by the authors.

3.3.1. Seed articles identification

We executed our search queries on Google Scholar and obtained an initial set of 4,599 articles. Then, we applied our general criteria, inclusion criteria, and exclusion criteria on these articles and retained

Table 7
Number of initial articles screened against the general, inclusion and exclusion criteria.

Qty.	General			Inclusion		Exclusion				
	G1	G2	G3	I1	I2	E1	E2	E3	E4	E5
3,988	Yes	Yes	No	Yes	No	Yes	No	No	No	No
457	Yes	No	No	Yes	No	Yes	Yes	No	No	No
34	Yes	Yes	Yes	Yes	No	Yes	No	No	No	Yes
81	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No
39	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No

Table 8
Number of snowballed articles screened against the general, inclusion and exclusion criteria.

Qty.	General			Inclusion		Exclusion				
	G1	G2	G3	I1	I2	E1	E2	E3	E4	E5
1,681	Yes	Yes	No	Yes	No	Yes	No	No	No	No
477	Yes	Yes	Yes	Yes	No	Yes	No	No	No	Yes
70	Yes	No	No	Yes	No	Yes	Yes	Yes	No	No
9	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No
82	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No

Table 9
Number of candidate articles screened against the general, inclusion, exclusion and quality assessment criteria.

Qty.	General			Inclusion		Exclusion					Quality			
	G1	G2	G3	I1	I2	E1	E2	E3	E4	E5	Q1	Q2	Q3	Q4
8	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No	Yes	No	No	Yes
5	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No	Yes	No	No
46	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No	No	Yes	Yes	Yes	Yes
62	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	Yes

99 articles. A breakdown of the number of articles in the initial set screened against the general, inclusion, and exclusion criteria is given in Table 7. We reviewed the title and abstract in all the 99 articles and divided those articles into two groups. The two groups are: (a) exclude the article, and (b) include the article. We identified 39 seed articles.

3.3.2. Candidate articles identification

We executed our search for candidate articles using forward snowballing and backward snowballing. We performed forward snowballing using Scopus and retrieved 1,097 articles that cited the 39 seed articles. We performed backward snowballing by retrieving all articles referenced in the 39 seed articles and obtained 1,222 articles. We collated these results and obtained 2,319 articles. Then, we applied our general criteria, inclusion criteria, and exclusion criteria on all the 2,319 articles and retained 82 articles. A breakdown on the number of snowballed articles screened against the general, inclusion and exclusion criteria is given in Table 8. To obtain the candidate articles, we combined the 82 articles and 39 seed articles and obtained 121 candidate articles.

3.3.3. Final articles identification

To identify the final set of articles, we read all the 121 candidate articles in detail and evaluated each article based on our general criteria, inclusion criteria, exclusion criteria, and quality assessment criteria. A breakdown of the number of candidate articles screened against the general, inclusion, exclusion, and quality assessment criteria is given in Table 9. We obtained the final set of 62 articles.

3.4. Data extraction

We reviewed all the 62 articles in detail and obtained data on the following:

- Algorithm, method or approach presented in a paper.
- The desired features of current SysLogCT approaches, the definition of those features and the classification criteria for those features.

We obtained data on the features of those SysLogCT approaches, collated and compared the data, and organized the data into several quality attributes. We used those quality attributes to build our quality model and evaluated the SysLogCT approach according to the identified quality attributes.

3.5. Data synthesis

We combined the data extracted from those SysLogCT approaches and summarized the data. A catalog of the relevant and representative SysLogCT approaches is presented in Section 5. Our evaluation of those SysLogCT approaches with respect to our quality model is given in Section 7.

3.6. Scope of this study

Our SLR considered SysLogCT approaches where their source code is available on the public domain, and SysLogCT approaches where their source code is not available due to their owner's distribution policy or software licence. We did not consider SysLogCT approaches for reliability, nor check the source code or implement the method to ascertain its correctness or complexity because it would require access to the code and significant resources beyond the scope of this paper.

4. Log-correlation workflow

In this section, we present the Log-Correlation workflow, describe six important quality attributes that SysLogCT approaches need to address, highlight the applications of cybersecurity log-analysis and summarize a number of publicly available cybersecurity datasets.

Fig. 2 shows a typical Log-Correlation workflow. The workflow is composed of three phases: (a) System logs reformatting, (b) Log-correlation, and (c) Log-analysis applications. The first phase of *System Logs Reformatting* converts unstructured or semi-structured log-messages into a standard format. The second phase of *Log-correlation* applies correlation algorithms or regression models to identify relationships in the standard formatted data for analysis. The third phase

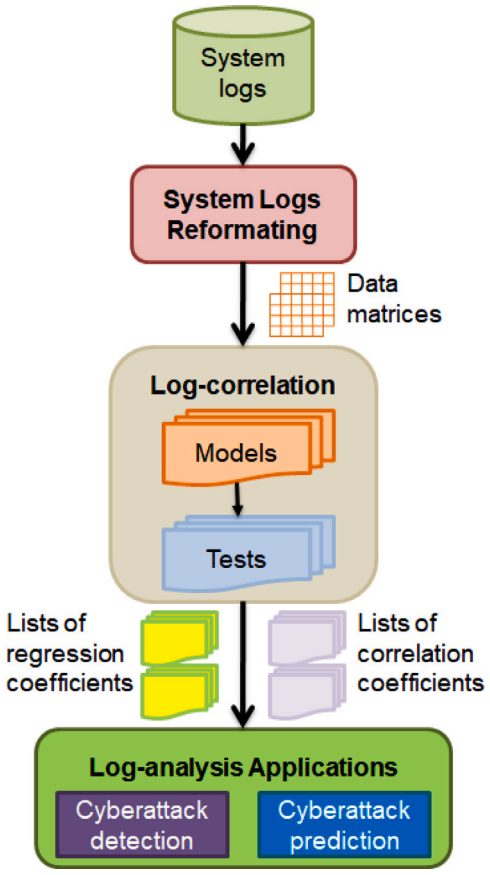


Fig. 2. The Log-Correlation workflow. It consists of a System Logs Reformatting phase, a Log-correlation phase and a Log-analysis Applications phase.

of *Log-analysis Applications* describes the applications of system log-analysis. Next, we describe details for each of the three phases in the system log-correlation workflow.

4.1. Phase 1: System logs reformatting

Most system log-messages contain three basic fields. Those fields are: (a) a time value, (b) a device number, and (c) a message. A parsed format of a NetFlow record obtained from an enterprise network operated by Los Alamos National Laboratories (LANL) [49,50], a Linux syslog [51] and a host event log-message also obtained from an enterprise network operated by LANL are shown in Fig. 3, Figs. 4 and 5, respectively.

The NetFlow data and Microsoft Windows host event logs contain one record per line. There is no mention of a specific NetFlow version in the Unified Host and Network Dataset. Hence, we obtained a description of all 11 fields in the NetFlow record from the Unified Host and Network Dataset published by LANL, which includes the source and destination ports [50]. The start time of the event in epoch time format (4302035) and the duration of the event in seconds (2297) are contained in the first and second fields, respectively. The source device (`Comp12013`) and destination device (`Comp584290`) identifiers are contained in the third and fourth fields, respectively. The network protocol number (6), source port number (`Port04167`) and destination port number (53) are contained in the fifth, sixth and seventh fields, respectively. In this NetFlow record, the destination port number 53 represents the Domain Name Service (DNS). The number of packets (359) and bytes (3948) sent by the source device are contained in the eighth and ninth fields, respectively. The number of packets (22976) and bytes (252672) sent by the destination device are contained in the tenth and eleventh fields, respectively. From Fig. 4, we observed that the Linux syslog contains a timestamp (`Jul 6 13:00:18`), a device number (`i106-308`) and a message (`[<% 0161lx>] do_page_fault 1189 2131`). From Fig. 5, we observed that this host event log-message contains the epoch time of the event in seconds (`"Time": 2`), source device (`"Source":"Comp309534"`) and an event identifier (`"EventID":4769`). Here, the event identifier 4769 represents a message that a kerberos service ticket was requested [50].

Table 10
A list of fields in the integrated log.

Field	Description
Starttime	The start time of the event.
Duration	The duration of the event.
Srcdev	The device which initiated the event.
Dstdev	The receiving device.
Message	The description of the event.
Srcpkts	The number of packets the "Srcdev" sent.
Dstpkts	The number of packets the "Dstdev" sent.
Srcbytes	The number of bytes the "Srcdev" sent.
Dstbytes	The number of bytes the "Dstdev" sent.
Srcport	The port used by "Srcdev".
Dstport	The port used by "Dstdev".
Proto	The network protocol number.
Usrname	The user account which initiated the event.
Svrname	The account name of a service the user is requesting for.
Status	The status of the request.
Procname	The name of the process executable.
Procid	The value that identifies a process.

The NetFlow record, Microsoft Windows log-message and Linux syslog contain a timestamp, a source device number and a message string. However, the content in the NetFlow reccord, Microsoft Windows log-message and Linux syslog are organized differently. In the NetFlow record and Microsoft Windows log-message, each field is delimited by a comma. In the Linux syslog, each field is delimited by a single space. Furthermore, there are no `EventID`, `UserName`, `ServiceName`, `DomainName`, `Status` and `Computer` fields in the NetFlow record and Linux syslog. The format of system logs and the content in those system logs in one network are different from the system logs generated on another network. This makes it difficult to parse the system logs, since a different parser needs to be implemented for each type of log. Additionally, while the use of correlation methods for identifying correlations of network events in the NetFlow data and correlations of other types of events in the Microsoft Windows logs or Linux syslogs separately is well known, the identification of correlations of network events and other types of events across the NetFlow data, Microsoft Windows logs and Linux syslogs is not common. However, as was mentioned at the start of this paragraph, most system logs contain a timestamp, a source device number and a message string. Therefore, the log-messages contained in the NetFlow data, Microsoft Windows logs and Linux syslogs can be integrated using these three fields. The integrated logs can be represented in a standard format containing fields from the NetFlow data, Microsoft Windows host event logs and Linux syslogs given in Table 10. A placeholder can be used to indicate a missing value or a field that is not present in the original log.

After the original logs are converted into the integrated log format, the fields and their associated values can be extracted. Most data analysis methods and machine learning algorithms use data matrices as their input [52,53]. A data matrix is a 2-dimensional data structure. The first dimension is represented by a column which corresponds to a field, e.g., `bytes sent by source device`. The second dimension is represented by a row which corresponds to an observation, e.g., `starttime`. Each cell in the data matrix contains a field value for an observation, e.g., the number of bytes sent by a source device at a starttime. Data matrices are useful for representing a large network. For example, an adjacency matrix is a square data matrix that represents a

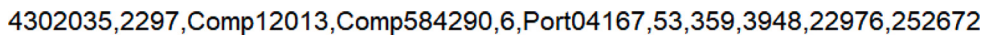


Fig. 3. A NetFlow record.

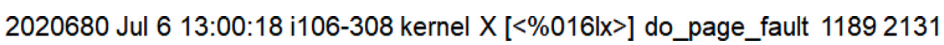


Fig. 4. A Linux syslog.

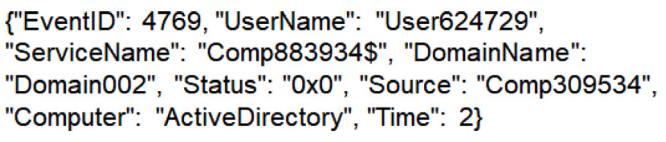


Fig. 5. A host event log-message.

finite graph and it is often used for modeling communications between devices on a network.

4.2. Phase 2: Log-correlation

The goal of Log-correlation is to determine the strength of the relationship between log-events, to group log-events which are strongly correlated, and to separate log-events which are weakly correlated or not correlated. SysLogCT approaches reduce the amount of log-events in the system logs, and the security analyst may use the list of correlated events to understand the relationship between multiple alerts [54]. For example, a threat actor may attempt to execute a port scan on multiple hosts and a typical user may also attempt to access a service provided by a host on the network. These network events can occur at the same time or at different times, and they can target random hosts. If the port scan carried out by the threat actor occurred on multiple hosts, the security analyst can obtain correlations of port requests on all hosts on the network to identify a smaller number of hosts for further analysis. However, if the port scan carried out by the threat actor targeted multiple ports on the same host, then pairwise correlation methods such as Pearson or Spearman-Rank correlation would not be able to identify port scans that occur at the same time on a single host [26].

In general, a SysLogCT approach consists of two components: (a) a model, and (b) a statistical test. The *Model* component applies a correlation algorithm or regression model on a given data matrix, and a list of correlation coefficients or a list of regression coefficients is generated. Once the correlation coefficients or regression coefficients are generated, the *statistical test* component uses statistical significance tests to validate the strength of the correlation coefficients or the regression coefficients. Then, a list of strongly positive correlated log-events is generated. However, a SysLogCT approach needs to address six important quality attributes, which we describe next.

4.3. Quality attributes of SysLogCT approaches

The six quality attributes SysLogCT approaches need to address are: (a) spurious relationships, (b) missing data values, (c) correlation threshold value, (d) bias and variance, (e) anomalies in the data, and (f) evaluation metrics.

4.3.1. Spurious relationships

A spurious relationship is a mathematical relationship in which two or more variables are strongly correlated, but those variables are not causally related, due to the presence of one or more confounding variables [55]. The security analyst must infer the likelihood that two or more log-events are associated with a cyberattack, which is difficult because some log-events can be strongly correlated due to coincidence, but one log-event does not actually cause the other log-events to occur. Consequently, these spurious relationships can lead to a wrong detection or a wrong prediction of a cyberattack.

4.3.2. Missing values in the data

Missing values in a dataset occurs when data for certain variables are not available. A malfunction in the data collection process or an error incurred during the data preprocessing stage are two of the most common causes of missing values. Missing values in the data can cause several problems [56]: (a) it can result in a misinformed or biased analysis and lead the security analyst to make a wrong conclusion, (b) it can decrease the ability of a statistical test in validating the strength of the correlation coefficients or regression coefficients, and (c) it can produce a biased estimate that leads to an incorrect result.

4.3.3. Correlation threshold value

A correlation threshold value is typically used to retain a smaller set of strongly positive correlated events for further analysis. However, the correlation threshold value is manually determined and this is a labor-intensive and time-consuming task. A security analyst must choose a threshold value and use it to identify a smaller set of log-messages or clusters of log-messages for further analysis. The Pearson or Spearman-Rank correlation coefficient (r-value) range from -1 to 1, where a r-value of -1 indicates a perfect negative correlation, a r-value of 0 indicates no correlation and a r-value of 1 indicates a perfect positive correlation. The standard rule of thumb for determining the strength of a relationship between two samples is [53]: (a) 0.1 to 0.3: weak positive correlation, (b) 0.31 to 0.69: moderate positive correlation, and (c) 0.7 to 0.99: strong positive correlation. Some security analysts may decide to select a small range of threshold values that range between 0.7 and 0.99 and use those values to identify events or clusters of events which are strongly correlated to a cyberattack [26,57].

4.3.4. Bias and variance

A penalty parameter is a single value used in regularized regression models such as the LASSO and Ridge to control the amount of bias and variance. A model that has a high bias will consistently make an incorrect detection or prediction. A model that has a high variance will make a correct detection or prediction inconsistently. The penalty parameters are important values that control a model's bias and variance. The penalty parameters are used to shrink a regression model's or a machine learning model's coefficients to prevent overfitting (i.e., reduce the amount of variance) or underfitting (i.e., reduce the amount of bias) the model to the training data [58]. If a fitted regression or machine learning model is too simple or complex, then the model will not predict the actual outcome accurately. Some recent works have developed novel techniques to address this issue by maximizing the coefficient-of-determination between the predicted values and actual values [59] or ranking the selection frequency of each independent variable in a regression model [60].

4.3.5. Anomalies in the data

An anomaly in the data is a value that differs significantly from the remaining data values. Anomalies in the data may increase or decrease the correlation coefficient or regression coefficient, depending on where those anomalies lay in the data. Data anomalies can: (a) impact the stability of the Pearson correlation coefficient [61], (b) overfit a regression model to the training data [62], or (c) influence the number of clusters of log-events [63]. To address this issue, several works have developed new methods that detected outliers such as robust correlation [64] and a robust estimator for linear regression problems [65].

4.3.6. Evaluation metrics

When a SysLogCT approach determined that a cyberattack had occurred, but no cyberattack actually occurred, this is called a false positive. When a SysLogCT approach determined that no cyberattack had occurred, but a cyberattack actually occurred, this is called a false negative. False positives and false negatives in SysLogCT approaches causes two problems [9]: (a) a false positive can consume the security analyst's time in investigating the security alert. and (b) a false negative can increase the risk of an attack on the network and reduces the security analyst's ability to respond to the attack.

4.4. Cybersecurity log-analysis applications and datasets

Several cybersecurity applications use the system logs to perform various analyses, e.g., detecting security alerts [66], preventing cyberattacks [67], constructing the chain of attacks [68] and managing user behavior [69]. In this section, we describe a number of cybersecurity log-analysis applications, discuss the influence of system log-analysis tools on those applications, and describe the cybersecurity datasets that are available in the public domain.

4.4.1. Cyberattack detection and prediction

Cyberattack detection tools are used to identify the times and sources of a current attack. These tools typically (a) use correlation techniques such as correlation-based K-nearest neighbor [70], Pearson correlation [32] or graph convolution networks [54] to obtain correlations of security alerts, or (b) select a small set of features in the system logs and use those features to train a detection model. Feature selection approaches such as Correlation-based Feature Selection (CFS) are widely implemented in cyberattack detection tools. Cyberattack prediction tools also use correlation techniques to identify correlations of security alerts or select a small set of features in the system logs using CFS. Unlike cyberattack detection tools, cyberattack prediction tools are used to identify the times and sources of a future attack or to predict the next step which an attacker will take. A SysLogCT approach is an important function in cyberattack detection and prediction tools because it is used to generate lists of correlated log-events needed for identifying the times and sources of current or future attacks, remove redundant features and select the most relevant features for training a detection model or a prediction model.

4.4.2. Correlated-based feature selection

Correlation-based Feature Selection is a feature selection technique that uses a correlation-based heuristic to identify a subset of relevant features [71]. Once the CFS algorithm identifies the relevant features, those features are used to train machine learning (ML) models to detect or predict a cyberattack. For example, Sarker [72] showed that using features with high Pearson correlation coefficients to train ML models increased the accuracy of those models. Heryanto et al. [73] also showed that the strongly correlated features increased the detection accuracy of their CFS-based intrusion detection system. Therefore, a SysLogCT approach provides an important function for increasing the accuracy of ML models through the removal of irrelevant features in the system logs.

4.4.3. Publicly available cybersecurity datasets

Several cybersecurity datasets including computer authentication logs, software process logs, network flow event logs, Domain Name Service (DNS) lookup logs, redteam compromise event logs, malware logs and industrial control system logs are available on the public domain to support research in cybersecurity system log-analysis. A summary of these cybersecurity datasets is given in Table 11. The Advanced Research in Cyber Systems at Los Alamos National Laboratories in the USA have created three datasets containing multiple cybersecurity events that ranged from 58 days to 9 months [49,50,74,75]. The Oak Ridge National Laboratory (ORNL) and University of Alabama (UoA) in Huntsville, USA have created three datasets that contain benign traffic and cyberattack events on industrial control systems [76,77]. The Canadian Institute for Cybersecurity (CIC) have provided multiple datasets that contain benign and malicious events ranging from malware attacks to Distributed Denial-of-Service (DDoS) attacks [78]. The Intelligent Security Group at University of New South Wales (UNSW) in Australia have created three Internet-of-Things (IoT) datasets that contain normal and botnet traffic [79–82]. The Cybersecurity research group at Robert Gordon University (RGU) in Aberdeen, UK have created a comprehensive attack dataset that contains both benign traffic and 26 real attacks on the controller area network of moving vehicles [83].

Table 11
Summary of publicly available cybersecurity datasets.

Org.	Dataset	Type of log
LANL, USA	1. Unified Host & Network Dataset	Network flow logs, Host event data.
	2. Multi-source cybersecurity events	Authentication logs, Process logs, Network flow logs, DNS lookup logs, Redteam events.
	3. User–Computer authentication	Authentication logs.
ORNL & UoA, USA	1. Power system	Audit logs
	2. Gas pipeline	Modbus logs
	3. Gas pipeline & water storage tank	Supervisory Control & Data Acquisition logs
CIC, Canada	1. Malware	Network flow logs
	2. DNS	Packet data
	3. Dark Web	System logs
	4. IoT	Network flow logs
	5. Language & model	System logs
UNSW, Australia	1. BoT–IoT	Network flow logs
	2. TON-IoT	Network flow logs, Linux syslogs, Host logs
	3. NB15	Network flow logs, PCAP files
RGU, UK	1. CAN Bus	System logs

5. SysLogCT approaches

In this section, we describe the relevant and representative SysLogCT approaches identified through our SLR. We reviewed the methods implemented by these SysLogCT approaches in detail and divided these SysLogCT approaches based on the class of methods. The classes of methods are: (a) graph methods, (b) machine learning methods, (c) statistical methods, and (d) hybrid methods that integrate one or more graph methods, machine learning methods, and statistical methods. The main categories of methods associated with these SysLogCT approaches are shown in Fig. 6.

5.1. SysLogCT approaches that developed graph methods

The following SysLogCT approaches implemented graph-based methods. Next, we describe the method developed by these SysLogCT approaches.

5.1.1. Real-time alert correlation based on code books

The Real-time Alert Correlation (RACC) is a novel method based on code books. RACC correlates alerts generated by an Intrusion Detection System (IDS) in real-time [14]. RACC is composed of two phases: (a) an offline data preprocessing phase, and (b) an online phase. In the offline phase, a set of data matrices are generated. The data matrices do not hold the actual alerts, but they contain keys to the corresponding alerts. An indexing technique based on red–black trees is implemented

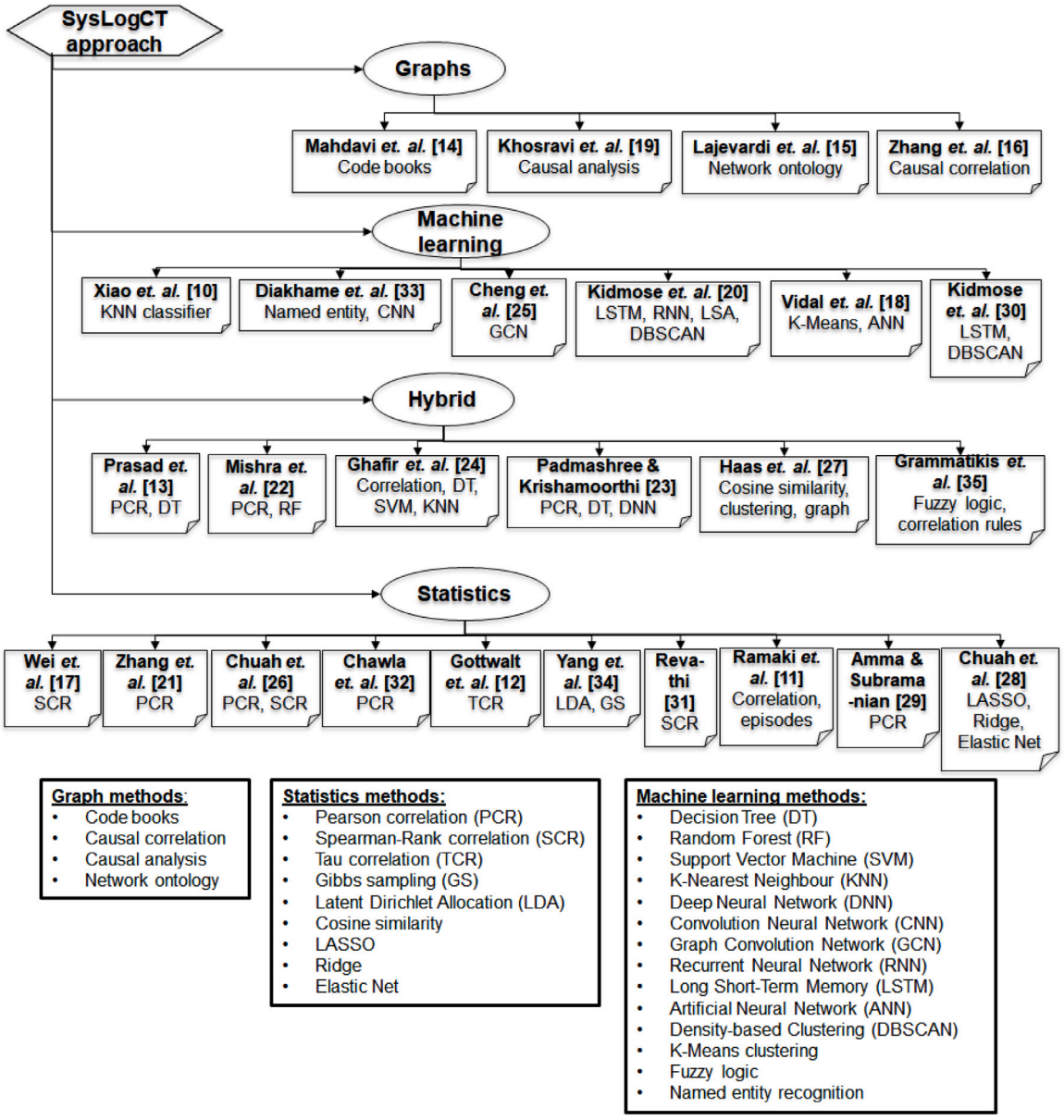


Fig. 6. Categories of methods associated with SysLogCT approaches.

to search the data matrices and modify the code book when an alert is generated. In the online phase, the red–black tree indexing technique and code book are integrated. The authors evaluated RACC on two cybersecurity datasets, compared the performance of their method with three alert correlation systems, and showed that RACC outperforms all the three alert correlation methods in terms of the average and maximum CPU processing times.

5.1.2. Alert correlation and causal analysis for detecting advanced persistent threats (APT)

Khosarvi and Ladani [19] introduced an approach that detected APT-based cyberattacks in real-time. Their approach is composed of four steps: (a) alert correlation, (b) meta-alert generation, (c) Intrusion Kill Chain (IKC) discovery, and (d) obtain the probability that a host is infected. In the alert correlation step, all alerts are modeled for each separate host, not for the whole system. In the meta-alert generation step, the authors defined a meta-alert as a 6-tuple that consists of a source port, a destination port, a source IP address, a destination IP address, an alert class and the time of the alert. Then, each alert is assigned to one or more of four IKC steps which includes reconnaissance & delivery, exploitation, operation, and data collection & exfiltration. In the IKC discovery step, a casual relationship between alerts is identified if all these conditions are satisfied: (a) alerts occur on the same host, (b) a cause alerts occurs before an effect alert, and (c) the alert class of a cause alert occurs before the alert class of an effect alert. In the host scoring step, two metrics are defined for estimating the likelihood that a host is infected. The first metric is called Time Impact Coefficient (TIC). TIC measures the time required to build a given IKC. An APT attack typically occurs over a long period of time. As such, the larger the TIC the more likely it is associated with an APT attack. The second metric is called Length Impact Coefficient (LIC). It contains the number of distinct meta-alerts. Then, the normalized attack surface of a host is calculated. When the normalized attack surface value of a host is high, it is more likely to be infected by an APT attack. They evaluated their approach on a cybersecurity dataset that contains real-world APT alerts, and showed that their approach detects various APT attacks with high recall, precision, accuracy and F1-scores and a low false positive rate.

5.1.3. Semantics-based correlation for detecting low-level and hybrid APTs

Lajevardi and Amini [15] proposed a new approach that correlated low-level operating system (O/S) events with network events to detect APTs. Their detection process consists of five steps: (a) intercepting events, (b) normalizing events, (c) correlating events, (d) detecting indirect access, and (e) checking for security policy violations. A system ontology is used to: (a) capture relations between O/S events such as file and memory mapped files and network events such as socket and I/O port, (b) detect relations between subjects such as host, process or user, (c) detect relations between subjects, O/S and network events, (d) generate a set of inference rules, and (e) capture security policy violations using inference rules generated. They generated several attack scenarios using available APT malware and evaluated their approach on attack scenarios generated. Their approach achieved high accuracy, high true positive and high true negative rates.

5.1.4. Detecting multistep attacks using alerts from smart grid monitoring systems

Zhang et al. [16] proposed a model that detected attacks in smart grids. Their model correlates IP addresses, constructs alert graphs, and transforms these alert graphs into candidate attack chains by implementing four steps: (a) preprocessing raw alerts, (b) building attack chains, (c) post-processing, and (d) building attack graphs. In the preprocessing step, raw alerts are converted into a 10-tuple and duplicate alerts are removed. In the attack chain building step, the source and destination IP addresses in the alert logs are used to construct candidate attack chains. Once the candidate attack chains are constructed, a child node aggregation algorithm is used to further reduce redundant alerts in the alert logs. In the post-processing step, several rules are defined to identify negative causal correlations between alerts in a candidate attack chain. When negative causal correlations are identified, irrelevant alerts in the candidate attack chain are removed. In the attack graph construction step, the attack chains are transformed into a directed graph. They evaluated their model on two cybersecurity datasets and showed that their method reduces the number of alerts to half the total number of alerts in the original alert logs.

5.2. SysLogCT approaches that developed machine learning methods

The following SysLogCT approaches implemented machine learning (ML) methods. Next, we describe the method implemented by these SysLogCT approaches.

5.2.1. A correlation-based KNN classifier for detecting distributed denial-of-service (DDoS) attacks

Xiao et al. [10] presented an effective approach that detected DDoS attacks. Their approach uses correlation-based filter selection method to reduce the size of the training data, trains the KNN classifier on the smaller training dataset and implements three modules. The first module is a flow collector that extracts traffic flows by inspecting the IP headers. The authors defined a flow as successive IP packets that contain the same source IP address, destination IP address, source port, destination port and protocol number. The second module is a feature selection module. It receives flows collected from the flow collector module and applies an existing probabilistic framework on bags of flows [84]. A bag of flows contains traffic flows which are generated by the same application. The probabilistic framework in [84] developed a new approach that modeled bags of flows based on Bayesian decision theory. The third module applies the KNN classifier on the correlation information obtained from the feature selection module and determines if a flow belongs to legitimate traffic or a DDoS attack. They evaluated their approach on three cybersecurity datasets, compared classification accuracy with three ML models, and showed their approach outperformed all three ML algorithms in terms of classification accuracy.

5.2.2. Correlating intrusion alerts to discover attack scenarios

Cheng et al. [25] proposed a framework called *Alert-GCN* that correlated intrusion alerts to discover attack scenarios. Their framework is based on the representation of alerts as graphs. *Alert-GCN* consists of three steps. Alerts that are generated by an IDS contain categorical data, which must be converted into numerical data before it can be used to train a machine learning model. In the first step, they addressed this issue by applying a technique called one-hot encoding, which assigns a numerical value to a categorical variable. In the second step, those alerts are transformed into an alert graph. The alert graph consists of: (a) nodes that represent individual alerts, and (b) edges that represent the similarity between two alerts. When the similarity of two alerts exceeds a specified threshold, an edge between these alerts is created. In the third step, a GCN is trained on the alert graph. They evaluated their framework on the DARPA 1999 cybersecurity dataset and showed that Alert-GCN outperformed seven ML algorithms in terms of precision, recall, F1-score and accuracy.

5.2.3. Detecting malware using packet payload analysis

Vidal et al. [18] introduced an alert correlation framework that detected malware by analyzing the network packet payload. Their framework consists of two layers: (a) detecting anomalies in the payload, and (b) extracting sequences of alerts. In the first layer, an Anomaly Diagnosis (AD) and a Nature Diagnosis (ND) component are developed to identify high risk malicious packets and reduce the number of false positives. The AD component obtains the difference between the packet's payload and a legitimate payload model. If the difference between the packet's payload and the legitimate payload model is large, then the probability that this packet contains malicious data is high. The K-Means clustering algorithm is implemented in the AD component and used to identify malicious packets. The ND component determines the level of risk for a malicious packet. It uses multiple rules to classify suspicious network packets. An ANN is implemented in the ND component and used to generate the rules. In the second layer, sequences of alerts are constructed using rules generated by the ND component. A genetic algorithm is implemented in the second layer and used to analyze sequences of alerts. The authors evaluated their framework on a new dataset that contains malicious and benign traffic payload data from the Data Center of the University Complutense in Madrid, Spain. Their results showed that their alert correlation framework classified alerts with high accuracy and identified a large number of false positives generated by anomaly-based intrusion detection systems.

5.2.4. An alert correlation framework for reconstructing cyberattack scenarios

Diakhame et al. [33] proposed a novel framework that reconstructed cyberattack scenarios. Their framework is composed of two phases: (a) reducing the number of alerts, and (b) correlating alerts. In the first phase, an existing algorithm that used Bayesian network algorithms [85] is employed to remove redundant alerts. Then a list of criteria is defined for merging multiple alerts into one alert. The criteria are: (a) alerts appear in the same time window, (b) alerts belong to the same category, and (c) alerts contain the same source and destination IP addresses. In the second phase, Named Entity Recognition (NER) and semantic similarity techniques are used to identify patterns of alerts. NER is employed to extract named entities including IP addresses, domain names, email addresses and hostnames contained in the alert. Then, a threshold check is performed on the extracted name entities to determine if two alerts share the same number of named entities. Once the threshold check is completed, a neural network technique called BERT [86] is used to extract the context associated with two alerts. The authors evaluated their framework on the UNSW-NB15 cybersecurity dataset. It contains multiple cyberattack scenarios and different categories of attacks including Denial-of-Service, worms, backdoor exploits and shellcode. They compared their framework with the cyberattack

detection algorithm in [85] and showed that their framework outperformed the cyberattack detection algorithm when correlating malicious packets with the correct attack scenario, and successfully reconstructed multiple cyberattack scenarios.

5.2.5. Correlating alerts on malware bot infections and identifying false intrusion alerts

Kidmose et al. [20] presented a novel method that determined which alerts are correlated. Their method is composed of three phases: (a) training a LSTM recurrent neural network, (b) detecting a correlation between two alerts, and (c) assigning new alerts to known incidents. In the first phase, pairs of alerts are input into the LSTM RNN to produce two output vectors. A similarity function based on cosine similarity is defined to obtain a similarity score for two output vectors. If the similarity score is equals to 1, then the two alerts are correlated. If the similarity score is not equals to 1, then the two alerts are not correlated. The similarity score for two alerts can range between 0 and 1. Thus, in the second phase, a threshold value of 0.5 is included. If the similarity score is greater than 0.5, then the two alerts are correlated. Multiple correlated alerts can be generated. Thus, in the third phase, the DBSCAN clustering algorithm is implemented and used to group alerts into clusters. A distance metric called *Epsilon* is used to determine if an alert belongs to a core cluster. If the alert is within the radius of a core cluster, then the alert is added to the cluster. If the alert is outside the radius of a core cluster, then the alert is classified as noise. The authors evaluated their method on a cybersecurity dataset that contains traces of benign traffic and malicious bot malware traffic. Their results showed that their method obtained a high average precision score and a high true positive rate.

Subsequently, Kidmose et al. [30] proposed a novel approach that integrated correlation and filtering techniques to identify correlated alerts and false intrusion alerts. Their approach which incorporates the LSTM RNN-based method developed by the same authors in [20] applies Latent Semantic Analysis on the text contained in alerts. Their LSA method counts the number of successive terms in an alert's text. Then, the Term Frequency Weighted by Inverse Document Frequency algorithm is used to determine the importance of a term within a group of alerts. They evaluated their approach on two cybersecurity datasets that contain benign and malicious malware bot traces, and showed that their approach achieved high accuracy, precision, recall and F1-scores on the two cybersecurity datasets. They defined a metric called *Incident Miss Rate* (IMR) that captures the likelihood that an incident will be missed. They obtained the IMR of their approach on the two cybersecurity datasets and showed that their approach achieved very low IMR rates on the two cybersecurity datasets.

5.3. SysLogCT approaches that developed hybrid methods

The following SysLogCT approaches implemented hybrid techniques that integrated graph, machine learning and/or statistical techniques. Next, we describe the approach implemented by these SysLogCT approaches.

5.3.1. A ML-based correlation analysis approach for detecting APT attacks

Ghafir et al. [24] proposed a novel approach called *MLAPT* that detected and predicted APT attacks. *MLAPT* consists of three main phases: (a) detecting threats, (b) correlating alerts, and (c) predicting an APT attack. In the first phase, eight modules were implemented. Those modules are designed to detect different APT attacks which include disguised executable files, malicious file hashes, malicious domain names, malicious IP addresses, malicious secure socket layer certificates, domain flux, scans and TOR connections. In the second phase, the output of each detection module is received and input into an alerts filter module, an alerts clustering module and a correlation indexing module. The alerts filter module receives all alerts generated by the detection modules and checks if a new alert is the same type and contains the same attributes as a recorded alert. If the new alert has the same type and the same attributes with a recorded alert, then the new alert is discarded. The alerts clustering module implements a four-step clustering algorithm that determines which alerts are associated with the same APT attack. The correlation indexing module implements a clustering algorithm that calculates a correlation value for two alerts based on the infected and scanned hosts attributes. In the third phase, four machine learning algorithms are applied on a preprocessed dataset to classify alerts. The machine learning algorithms are Decision Trees, Support Vector Machine, K-Nearest Neighbor and Ensemble Learning. The authors evaluated *MLAPT* on a simulated dataset that contains 100 full APT attack scenarios, 100 sub-APT attack scenarios, and 4250 random alerts. Their results showed that *MLAPT* achieved an accuracy that ranged between 65% and 90% on the full APT and sub-APT attacks, and achieved a low false positive rate that ranged between 1% and 1.6%.

5.3.2. CFS techniques for early detection of DDoS attacks

Mishra et al. [22] implemented a technique that performed early detection of DDoS attacks in an IoT environment. Their technique extracts important features in the data and employs multiple statistical and machine learning methods. Three phases are developed in their technique. The first phase is Data preprocessing. It removes missing values in the data, uses label encoding to convert categorical variables into integers or floats, and all the data values are scaled to range between 0 and 1. The second phase is Feature selection. It applies Pearson correlation, Information Gain and Kendall Tau Rank correlation to extract the most relevant features. The third phase is Classification. It applies Random Forest, Decision Tree, XGBoost and AdaBoost to classify malicious events. The authors evaluated their technique on the BoT–IoT cybersecurity dataset. They showed that when Decision Tree, Random Forest, XGBoost and AdaBoost algorithms are trained using features selected by Pearson correlation and Kendall Tau Rank correlation, all four ML algorithms achieved high accuracy, precision, recall and F1-scores.

5.3.3. Identifying attack scenarios using network motifs

Haas et al. [27] presented an approach that identified known attack scenarios and derived new signatures for unknown attack scenarios. Their approach consists of two phases: (a) graph construction, and (b) attack classification. In the first phase, alerts are transformed into a Communications Structure Graph (CSG). An alert is defined as a four-tuple that consists of a source IP address, source port, destination IP address and destination port. Once the CSG is generated, all subgraphs in the CSG are assigned to specific motifs. The number of times a motif occurs is calculated and a motif signature is produced. However, the size of a motif's signature depends on the size of the graph. Consequently, the signature of two motifs cannot be directly compared. To solve this problem, the authors used a metric called *Z-Score* [87]. The *Z-Score* is defined as the difference in the number of absolute motifs and the average absolute count of motifs in a random graph divided by its standard deviation. The Cosine similarity measure is adapted to obtain the similarity score for two motifs. In the second phase, a signature-based classification technique is used to classify attack scenarios. The technique maps motifs to reference attack scenarios. However, it can only classify alerts to known attacks. To map alerts to a new attack scenario, an unsupervised clustering technique is developed. The unsupervised clustering technique is composed of two steps: (a) cluster alerts, and (b) derive reference attack scenarios. Hierarchical clustering is used to cluster similar alerts based on their motif signatures. Then, a motif signature is derived for each cluster and used as a reference scenario. To evaluate their approach, the authors designed two experiments. The first experiment ascertains if motif signatures can be used for identifying and comparing different attack scenarios. The second experiment evaluates their approach on real-world cybersecurity data. In the first experiment, a synthetic dataset

containing alerts with ground-truth labels is generated. They evaluated their approach on the synthetic dataset and obtained a very high accuracy. In the second experiment, the DShield dataset containing 34,204 attacks is used [88]. They evaluated their approach on the DShield dataset and classified 76% to 96% of attacks correctly.

5.3.4. A security information and event management (SIEM) system for smart grids

Grammatikis et al. [35] presented a SIEM system called *SPEAR* that detected and correlated cyberattacks and anomalies in Smart Grids. *SPEAR* is composed of three layers: (a) a data capture layer, (b) a detection layer, and (c) a correlation layer. In the data capture layer, statistics on network flows, information about packet payloads and operational data are captured using multiple sensors. Then, the data is sent to the detection layer which executes four intrusion detection processes. Those intrusion detection processes consist of network flow-based detection, packet-based detection, operational data-based detection and visual-based detection. The output from the detection layer are security events detected. Once the security events are detected, those events are sent to the correlation layer. The correlation layer executes two processes. In the first process, a set of correlation rules based on the data features are defined. Those rules are applied to security events to determine if two or more security events are correlated. In the second process, a reputation value is calculated for each Smart Grid asset. A set of rules based on fuzzy logic is defined. Those rules are applied on security events to obtain a severity level for each security event. Multiple variables including the asset's value, event risk, event priority and event subcategory are used in the calculation. The authors created multiple new datasets that contain benign and malicious events for four use cases. Those use cases consist of a hydropower plant, an electrical substation, a power plant and a smart home. They evaluated *SPEAR* on their datasets and showed that all the four intrusion detection processes achieved a high accuracy score, a high true positive rate, a high F1-score and a low false positive rate.

5.3.5. A multi-level CFS method for IDS

Prasad et al. [13] analyzed the UNSW-NB15 cybersecurity dataset and proposed a CFS method that detected attacks in the UNSW-NB15 dataset. Their method is composed of three stages: (a) data normalization, (b) correlation-based feature selection, and (c) classification. In the first stage, all values in the data are rescaled to range between 0 and 1. The data values are rescaled to improve the performance of a machine learning classifier. In the second stage, the Pearson correlation algorithm is used to obtain the correlation values of features in the data. Then, the correlation values are ordered in descending order and ranked. The highest ranked features are selected based on the correlation value's strength. In the third stage, the Decision Tree classifier is used to classify malicious events in the UNSW-NB15 dataset. To reduce the number of features for training the DT classifier, one feature from a pair of features where the correlation value is greater than or equal to 0.9 is selected. The authors compared their method with six CFS methods and showed that their CFS method outperformed those six methods in terms of the accuracy, the detection rate and the false positive rate.

5.3.6. A novel CFS method for detecting intrusions in IoT networks

Padmashree and Krishnamoorthi [23] proposed a CFS method that detected intrusions in IoT networks. Their method is composed of three stages: (a) data preprocessing, (b) feature extraction, and (c) classification. In the first stage, data cleaning, data normalization and one-hot encoding are applied on the data. First, the data is cleaned by removing duplicate values. Then, all values in the data are rescaled to range between 0 and 1. One-hot encoding is used to convert categorical variables into numerical values. In the second stage, a novel feature selection technique called *DT-PCRFE* is implemented. It is based on the information entropy index generated by the Decision Tree algorithm. *DT-PCRFE* uses the information entropy index to calculate a score value for each feature. Once the score values are produced, the Pearson correlation algorithm is applied on the score values and the correlation coefficient for two features is obtained. A threshold value is specified and used to rank the correlated features. In the third stage, a DNN algorithm is implemented. The DNN algorithm is configured with the Rectified Linear Unit function for its hidden layers and the softmax activation function for its output layer. The authors evaluated their CFS method on the BoT–IoT dataset provided by the University of New South Wales in Australia. Their results showed that their CFS method classified attack events and normal events with a high accuracy, high true positive, precision and recall scores, and a low false positive rate. Furthermore, they compared their method with four CFS methods and showed that their method outperformed the four CFS methods in terms of accuracy, false positive rate, precision, recall and F1-scores.

5.4. SysLogCT approaches that developed statistical techniques

The following SysLogCT approaches implemented statistical techniques. Next, we describe the technique implemented by these SysLogCT approaches.

5.4.1. Rank-correlation based reflection DDoS detection

Wei et al. [17] proposed a rank-correlation based algorithm that detected reflection DDoS attacks. A standard DDoS attack overwhelms the network by generating traffic from multiple sources without using an intermediary. In contrast to a standard DDoS attack, a reflection DDoS attack uses an intermediary to generate a large volume of traffic and masks the attacker's identity. Their proposed method implements the Spearman-Rank correlation algorithm. The Pearson correlation algorithm is one of the most widely used correlation method. However, the Pearson correlation algorithm is susceptible to bursts of network traffic and outlier values in the data. Unlike the Pearson correlation algorithm, the Spearman-Rank correlation algorithm converts raw values into ranked values and orders the ranked values in ascending order. Then, the covariance of two values are normalized by their standard deviation. As such, the Spearman-Rank correlation algorithm is less susceptible to outlier values. The authors evaluated their method on two reflection DDoS attack scenarios using the NS2 network simulation system. Their results showed that their method achieved low false negative rates and low false positive rates.

5.4.2. A correlation-based method for analyzing the Code Red worm

Zhang et al. [21] proposed a correlation-based method that analyzed how the Code Red worm spreads in a network. Code Red is an Internet worm that exploited a buffer overflow error in the data indexing extension of Microsoft Internet Information Services (IIS) server. When an IIS server is infected, random IP addresses are generated, and the Code Red worm tries to replicate itself onto machines that map to those IP addresses. Their correlation-based method implements the Pearson correlation algorithm. The Pearson correlation coefficient (r) ranges between -1 and 1. When $r = 1$, it indicates that the two variables are perfectly positively correlated. When $r = 0$, it indicates that the two variables are not correlated. When $r = -1$, it indicates that the two variables are perfectly negatively correlated. They evaluated their method on a dataset containing NetFlow records associated with the Code Red worm, and showed that more than 90% of correlation coefficients are above 0.9, indicating that their correlation-based method can identify many anomalies.

5.4.3. Challenges in identifying network attacks in the NetFlow data

Chuah et al. [26] applied a correlation approach on unlabeled NetFlow data from a large enterprise network, and showed that reflection attacks that were executed on the Secure Shell Host (SSH) servers were not correlated, and the reflection attacks on the Domain Name System (DNS) servers were also not correlated. Their approach used Pearson and Spearman-Rank correlation, and is composed of three main phases: (a) correlating the times in the NetFlow records, (b) correlating the ports in the NetFlow records, and (c) obtaining the difference in time between the NetFlow records. Pearson correlation is widely used to detect network attacks [32,73,89]. While Pearson correlation has been shown to be effective in detecting network attacks, it can only identify correlations between two network events. Furthermore, several correlated network events can be produced, and these correlated network events must be manually analyzed before an attack can be identified. To address the two limitations of the Pearson correlation approach, Chuah and Suri [28] evaluated three regularized regression models. They analyzed 1.7 million NetFlow records from the same enterprise network and showed that (a) reflection attacks on the Network Time Protocol servers were not correlated, and (b) reflection attacks on the NetBIOS servers were also not correlated.

5.4.4. A novel framework for detecting DDoS attacks and flash events

Chawla et al. [32] proposed a novel framework called *CoDFEDD* that detected DDoS attacks and flash events. Their framework is based on the Pearson correlation algorithm. *CoDFEDD* is composed of two phases: (a) a detection phase, and (b) a discrimination phase. In the first phase, the network traffic is analyzed and the normal traffic is separated from abnormal traffic. In the second phase, the abnormal traffic is further analyzed to distinguish DDoS traffic from flash event traffic. *CoDFEDD* is implemented through five modules. The Flow Collector module aggregates network packets from routers. The Feature Extractor module extracts features in the packet header. The Anomaly Detector module analyzes incoming network flows and raises an alarm when an anomaly is detected. The Flow Estimator module computes the Pearson correlation coefficient between pairs of network flows. The correlation coefficient (r) is used to obtain a value for CoDFEDD. If r is greater than a specified threshold value, then the CoDFEDD value is set to 1. If r is less than the threshold value, then the CoDFEDD value is set to 0. The Decision Maker module uses the CoDFEDD value to determine if an abnormal network flow is a DDoS attack or a flash event. The authors evaluated their method on three cybersecurity datasets. Their results showed that: (a) network flows associated with DDoS attacks are strongly correlated and the CoDFEDD value is close to 1, and (b) network flows associated with flash events are weakly correlated and the CoDFEDD value is below 1.

5.4.5. A multivariate correlation-based feature selection method for network anomaly detection

Gottwalt et al. [12] introduced a feature selection method called *CorrCorr* that detected network anomalies. *CorrCorr* is based on multivariate correlation techniques, and it is implemented by three modules: (a) data extraction and preprocessing, (b) feature selection, and (c) model training and testing. In the data extraction and preprocessing module, the raw data is transformed into a form compatible with correlation techniques. The raw data contains binary and categorical features which must be converted into numerical values. The binary and categorical features are converted into a set of ordered numbers. Then, the Pearson correlation algorithm is applied on all features to identify redundant features. If the Pearson correlation coefficient is 0.8 or above, those features are removed. Once redundant features are removed, the remaining features are normalized to remove bias. In the feature selection module, the Pearson correlation algorithm and Principal Component Analysis (PCA) method are implemented. The feature selection process is composed of three steps: (a) calculate correlations of features, (b) calculate the symmetric tau (τ) to obtain the feature relevance [90], and (c) rank all features based on the strength of their correlation scores. In the model training and testing module, the k-fold cross validation method is implemented. The preprocessed data is randomly divided into 10 bins containing equal number of samples. One bin is used as the test set and nine bins are used as the training set. 3 to 20 relevant features are used to evaluate *CorrCorr*'s performance. The authors evaluated *CorrCorr* on the UNSW-NB15 and NSL-KDD cybersecurity datasets. They compared *CorrCorr* with Pearson correlation and PCA, and their results showed that *CorrCorr* outperformed Pearson correlation and PCA in terms of accuracy, detection rate and false positive rate.

5.4.6. An efficient framework for correlating alerts in early warning systems

Ramaki et al. [11] proposed an efficient framework called *RTECA* that correlated alerts in early warning systems. *RTECA* is composed of five components: (a) alert aggregation, (b) episode extraction, (c) learning and detection, (d) prediction, and (e) new attack mining. In the alert aggregation component, all alerts belonging to the same type and satisfying a predefined time window are merged into hyperalerts. In the episode extraction component, the authors defined an episode as a partially ordered set of hyperalerts that can be represented by a directed acyclic graph. A serial episode is extracted by sliding a batch window over a sequence of episodes. The authors defined a critical episode as a frequent serial episode when its correlation strength exceeds a predefined threshold. Then, the authors defined a benign episode as a frequent serial episode where its correlation strength is below the predefined threshold. In the learning and detection component, attack trees are constructed by identifying the critical episodes. In the prediction component, knowledge of attack scenarios are used to predict the next step in the attack scenario. In the new attack mining component, sub-episodes in the benign episodes are extracted. To distinguish false positive sub-episodes from sub-episodes associated with a new attack scenario, a set of rules are defined. If a benign sub-episode satisfies all those rules, it is labeled a malicious episode. Those rules are: (a) a new alert type is detected, (b) the similarity score for the source IP addresses of two alerts exceeds a predefined threshold, (c) the similarity score for the destination IP addresses of two alerts exceeds the same predefined threshold, and (d) the severity score for the alert exceeds a separate predefined threshold. The authors evaluated *RTECA* on two real-world cybersecurity datasets. Their results showed that *RTECA* identified the malicious episodes with a very high accuracy on the two cybersecurity datasets.

5.4.7. A rank-correlation method for detecting reflection DDoS attacks

Revathi [31] proposed an algorithm called *FRCD* that detected reflection DDoS attacks. *FRCD* is composed of five steps: (a) monitor network flows, (b) locate suspicious flows, (c) calculate rank correlation, (d) calculate flow correlation, and (e) identify reflection DDoS attack. In the first step, the network packet's content is inspected. The content is passed to the suspicious flow locator which will raise an alarm if the packet is found to be suspicious. In the third and fourth steps, the Spearman-Rank correlation algorithm is applied on network flows to ascertain if those flows are associated with a reflection DDoS attack. Two threshold values are specified. If the correlation coefficient of two network flows is between -0.15 and 0.3, then the two network flows are labeled benign. If the correlation coefficient of two network flows is close to 1, then those flows are labeled suspicious. In the fifth step, network flows which belong to flash crowds and network flows which are actual reflection DDoS attacks are distinguished. The author evaluated *FRCD* on a network simulation and showed that their algorithm achieved a network efficiency performance of 99.86%.

5.4.8. An APT detection method based on causality correlation and semantic analysis

Yang et al. [34] proposed a system called *Poirot* that detected multi-stage threats over long duration using alerts from existing intrusion detection systems. *Poirot* constructs alert chains by automatically mining causes between anomalies. Furthermore, it models the semantic context of alert chains using a semantic analysis method called Latent Dirichlet Allocation (LDA). *Poirot* is composed of four modules: (a) a data collection module, (b) a preprocessing module, (c) a causality analysis module, and (d) a semantic analysis module. In the first module, data from multiple sources including but not limited to the operating system, traffic detection system, honeypot system and IDS are collected and collated into system logs. Then, those system logs are input into the second module. The second module converts all events in the system logs into a standard security event format called Intrusion Detection Message Exchange Format (IDMEF). The authors defined a triple that contains: (a) a name that describes the event, (b) the event's IP address, and (c) the time that the event was detected. Then, all alerts are rearranged in chronological order. The third module generates a causality matrix. To obtain a causality score for two alerts, the authors defined a metric called *Average Causal Effect* (ACE). ACE is the difference between the expected mean of two variables. They applied ACE on all pairs of alerts. In the fourth module, LDA is used to construct chains of alerts. Gibbs sampling is applied on the alert chain and an estimate of the likelihood that the alert chain is a multi-stage attack is obtained. The authors evaluated *Poirot* on 29 security logs containing traces of Trojan horse, backdoor traffic, remote code execution, common gateway interface attacks, directory traversal attacks and IPV4 sweep events, and showed that *Poirot* achieved very high recall and accuracy and a low false positive rate. Furthermore, they compared *Poirot* with an email spam filter and an existing APT detection method called CONAN [91]. Their results showed that *Poirot* outperformed the email span filter and CONAN in terms of accuracy, precision, recall and F-score.

5.4.9. A feature correlation map method for detecting denial-of-service (DoS) attacks

Amma and Subramanian [29] proposed a system that uses Feature Correlation Maps (FCM) for detecting DoS attacks. Their system is composed of four steps: (a) select the relevant features, (b) generate a feature correlation map, (c) construct profiles for normal traffic, and (d) detect a DoS attack. In the first step, a linear correlation technique is implemented. First, the covariance of a feature and the class label is calculated. Second, the variance for the feature and variance for the class label are multiplied and the square root is then taken. Third, the covariance of the feature and class label is divided by the square root of the feature variance multiplied by the class label variance. Features with correlation coefficients greater than a predefined threshold are selected. In the second step, the distance between two selected features is calculated. Then, a feature correlation map containing the distance for all pairs of selected features is constructed. In the third step, the Hellinger distance measure is applied to the feature correlation map and profiles of normal network traffic are obtained. In the fourth step, a threshold value is defined to detect anomalies. This threshold value is obtained by adding the mean Hellinger Distance (HD) for normal traffic data and the standard deviation HD for normal traffic data, and then multiply it by a significance value that range from 1 to 3. Then, the HD between the test data FCM and the normal data FCM is obtained. If the HD for the test and normal data exceeds the threshold value, the network traffic is identified as a DoS attack. The authors evaluated their system on three cybersecurity datasets. Their results showed that their system achieved a high accuracy score, a high true positive rate, and a low false positive rate on all the three cybersecurity datasets. Furthermore, they compared their system with three existing DoS detection methods. Their results showed that their system outperformed the three DoS detection methods in terms of accuracy and false positive rate on all the three cybersecurity datasets.

6. Quality model for SysLogCT approaches

In this section, we present our quality model for evaluating SysLogCT approaches. In [92], it is stated that a quality model is either quantitative or qualitative. A quantitative quality model is composed of attributes that are directly measurable. In contrast, a qualitative quality model is composed of attributes which are not directly measurable. In [44], the authors identified four important quality attributes that SysLogCT approaches that diagnosed or predicted failures in cluster systems should have. These quality attributes include spurious correlation, correlation threshold settings, missing data, and outliers in the data. In [93], the authors showed that the choice of penalty parameters influences the goodness-of-fit of a regression model and the models' predictive accuracy. In [94], the authors showed that the false positive rate and false negative rate are important metrics for evaluating the success of machine learning classifiers. Therefore, we propose a qualitative quality model and define six quality attributes which we identified from our systematic literature review. The quality attributes are: (a) spurious relationships, (b) missing data values, (c) correlation threshold value, (d) bias and variance, (e) anomalies in the data, and (f) evaluation metrics. We evaluate our quality model on the identified SysLogCT approaches and the results are presented in Section 7. Next, we describe details for all the six quality attributes that consist of a typical question, a definition, classification criteria, and a domain.

6.1. Spurious relationships

A SysLogCT approach can incorrectly detect or predict a cyberattack when spurious relationships are present in the data.

6.1.1. Typical question

If multiple network events appear to be causally related, but their relationship did not cause a cyberattack, can a SysLogCT approach determine that a spurious relationship exists between those network events?

6.1.2. Definition

Simon [55] defined a spurious relationship as two or more variables that are strongly correlated but not causally related due to coincidence or due to the presence of one or more confounding variables.

6.1.3. Classification criteria

We studied the method implemented in a SysLogCT approach. If a SysLogCT approach did not detect spurious relationships between two or more network events, it is classified as "Likely cause". However, if a SysLogCT approach detects spurious relationships between network events, it is classified as "Potential true cause".

6.1.4. Domain

The domains are: (a) Likely cause, and (b) Potential true cause.

6.2. Missing data values

Missing values in the data can decrease the ability of a SysLogCT approach in validating the strength of a correlation coefficient. It can also cause a SysLogCT approach to produce a biased estimate that does not lead to correct detection or prediction of a cyberattack.

6.2.1. Typical question

If multiple network events are missing in the NetFlow data, can a SysLogCT approach still detect or predict a cyberattack accurately?

6.2.2. Definition

Graham [95] defined missing data as "*a data value that is not stored for a variable in an observation of interest.*" When a dataset contains several empty cells, there are two problems that need to be addressed: (a) reduced statistical power, and (b) bias in selecting features. In general, a reduction in statistical power occurs when the null hypothesis is incorrectly rejected. Selection bias occurs when a subset of samples in the data is excluded from the study. Data can be excluded due to several issues, such as a failure in the data collection process or the inappropriate use of techniques that handle missing data. For example, Pawlicki et al. [96] showed that the accuracy of the Random Forest classifier was significantly reduced when only fully described data points were used.

6.2.3. Classification criteria

We studied the method implemented in a SysLogCT approach. If a dataset that contains empty cells is used to evaluate a SysLogCT approach and a process that handles missing network events is used by the SysLogCT approach, this is classified as "Robust against missing network events". If a dataset that contains empty cells is used but the SysLogCT approach did not use a process for handling missing network events, this is classified as "Potential to be robust against missing network events". If the SysLogCT approach did not use a data imputation technique, and it is not evaluated on a dataset that contains empty cells, this is classified as "Not robust against missing network events".

6.2.4. Domain

The domains are: (a) Robust against missing network events, (b) Potentially robust against missing network events, and (c) Not robust against missing network events.

6.3. Correlation threshold value

The correlation threshold value is used to identify a smaller set of network events for further analysis, but determining the optimal correlation threshold value is a labor-intensive and time-consuming task.

6.3.1. Typical question

What techniques are implemented in a SysLogCT approach to select an optimal correlation threshold value?

6.3.2. Definition

A correlation threshold is a value that is typically used to: (a) identify features which are strongly correlated, and (b) remove redundant features. For example, Gottwalt et al. [12] specified a fixed threshold value of 0.8 and used it to remove features when the Pearson correlation coefficient obtained is equal to or above the threshold value. On the other hand, as an example, Grammatikis et al. [35] defined a set of correlation rules and those rules were used to identify strongly correlated SIEM alerts.

6.3.3. Classification criteria

We studied how threshold values are determined in a SysLogCT approach. If a fixed, specified threshold value or a threshold criterion is implemented in a SysLogCT approach, it is classified as "Fixed'. If policies or rules, multiple threshold parameters or a range of threshold values is implemented in a SysLogCT approach, it is classified as "Adjustable". If a threshold value is not mentioned in a SysLogCT approach, it is classified as "Not specified".

6.3.4. Domain

The domains are: (a) Fixed, (b) Adjustable, and (c) Not specified.

6.4. Bias and variance

A SysLogCT approach will consistently produce a wrong detection or prediction of a cyberattack when using a regression or machine learning (ML) model with a high bias. Additionally, a SysLogCT approach will produce a correct detection or prediction of a cyberattack inconsistently when using a regression or ML model with a high variance.

6.4.1. Typical question

What techniques are implemented in a SysLogCT approach to reduce the amount of bias and variance?

6.4.2. Definition

In machine learning, bias is defined as the difference between a value predicted by a model and the observed value in the data [58]. Variance is defined as the variability of a model's predicted value [58]. If a SysLogCT approach did not reduce the amount of bias and variance, it can produce a detection or prediction model that performs well on the training data but performs poorly on unseen data.

6.4.3. Classification criteria

We studied how bias and variance are addressed in a SysLogCT approach. If a SysLogCT approach: (a) implemented an ML model and used k-fold cross-validation to train and test the ML model, or (b) implemented a regularization technique, it is classified as "Potentially robust against bias and variance". If a SysLogCT approach implemented an ML model and divided the data into one training set and one testing set, it is classified as "Likely robust against bias and variance". If a SysLogCT approach did not implement an ML model, it is classified as "Not specified".

6.4.4. Domain

The domains are: (a) Potentially robust against bias and variance, (b) Likely robust against bias and variance, and (c) Not specified.

6.5. Anomalies in the data

Outlier values in the data can skew the correlation coefficient produced by a correlation algorithm. Outlier values can also cause a regression model or ML model to overfit on the training data and produce a model with high variance.

6.5.1. Typical question

What techniques are implemented in a SysLogCT approach to detect and handle outlier values in the data?

6.5.2. Definition

In data analysis, an anomaly is defined as an event or an observation that deviates significantly from the majority of events or observations in the data [52]. Wei et al. [17] compared the Pearson correlation and Spearman-Rank correlation based feature selection methods and showed that the detection accuracy for reflection DDoS attacks is increased when the Spearman-Rank correlation based feature selection method is used. If a SysLogCT approach implemented a correlation technique that is susceptible to anomalies in the data or did not standardize all values in the data, it may lead to reduced detection or prediction accuracy of cyberattacks.

6.5.3. Classification criteria

We studied the method implemented in a SysLogCT approach. If a SysLogCT approach: (a) contains a statistical correlation method such as Spearman-Rank correlation, Kendall Tau correlation or a statistical-heuristic feature selection method [90], or (b) contains a statistical correlation method such as Pearson correlation and scales all values in the data, it is classified as "Robust against anomalies". If a SysLogCT approach contains a statistical correlation method such as Pearson correlation and did not scale all values in the data, it is classified as "Susceptible to anomalies". If a SysLogCT approach did not implement a statistical correlation method such as Pearson correlation, Spearman-Rank correlation or Kendall Tau correlation, it is classified as "Not Specified".

6.5.4. Domain

The domains are: (a) Robust against anomalies, (b) Susceptible to anomalies, and (c) Not Specified.

6.6. Evaluation metrics

The effectiveness of a SysLogCT approach in detecting or predicting a cyberattack depends on its ability to reduce the number of false alerts and missed attacks.

6.6.1. Typical question

What evaluation metrics are used to evaluate the effectiveness of a SysLogCT approach in reducing the amount of false alerts and missed attacks?

6.6.2. Definition

In data mining, a false positive is defined as a result that has been determined to be true when it is actually false. A false negative is defined as a result that has been determined to be false when it is actually true. A robust metric provides insights into how well a SysLogCT approach performs and enables comparing similar SysLogCT approaches. The standard FPR metric is given as $FP \div (TN + FP)$, where FP is the number of false positives and TN is the number of true negatives [52]. The standard FNR metric is given as $FN \div (TP + FN)$, where FN is the number of false negatives and TP is the number of true positives [52]. Furthermore, metrics such as accuracy, precision and recall are used to evaluate a SysLogCT approach that detects or predicts events whose values are categorical. Conversely, metrics such as the Mean Squared Error (MSE) or coefficient-of-determination (R^2) are used to evaluate a SysLogCT approach that detects or predicts events whose values are real numbers [97].

6.6.3. Classification criteria

We studied the metrics used to evaluate the FPR and FNR of a SysLogCT approach. If evaluation metrics that determined the false positive rate and false negative rate are used, it is classified as "Evaluated *wrt.* false positive rate and false negative rate". If evaluation metrics that determined the false positive rate are used, it is classified as "Evaluated *wrt.* false positive rate". If evaluation metrics that determined the false negative rate are used, it is classified as "Evaluated *wrt.* false negative rate".

6.6.4. Domain

The domains are: (a) Evaluated *wrt.* false positive rate and false negative rate, (b) Evaluated *wrt.* false positive rate, and (c) Evaluated *wrt.* false negative rate.

7. Comparison of SysLogCT approaches

In this section, we organize our comparison of SysLogCT approaches reviewed in Section 5 according to the six quality attributes of spurious relationships, missing data values, correlation threshold value, bias and variance, data anomalies and evaluation metrics described in our quality model (see Section 6). We obtained our observations by conducting a thorough review of the methodology and results described in the articles shown in Fig. 6 and detailed in Section 5. Furthermore, we evaluate our quality model by comparing these SysLogCT approaches with respect to all the six quality attributes. A summary of the results from our quality model is given in Table 12. We identified many promising directions for future research that could improve SysLogCT approaches in detecting or predicting cyberattacks. These future research directions are discussed according to each of the six quality attributes.

7.1. Spurious relationships

From Table 12, we observed that Xiao *et al.* [10], Ramaki et al. [11], Gottwalt et al. [12], Prasad et al. [13], Mahdavi et al. [14], Lajevardi et al. [15], Zhang et al. [16], Wei et al. [17], Vidal et al. [18], Khosravi et al. [19], Kidmose et al. [20,30], Zhang et al. [21], Mishra et al. [22], Padmashree & Krishnamoorthi [23], Ghafir et al. [24], Cheng et al. [25], Chuah et al. [26,28], Haas et al. [27], Amma & Subramanian [29], Revathi [31], Chawla et al. [32], Diakhame et al. [33], Yang et al. [34] and Grammatikis et al. [35] developed approaches that performed direct correlation of system log-events to various attack scenarios.

Mahdavi et al. [14] evaluated RACC on the DARPA2000 cybersecurity dataset and a private dataset that contains alerts generated by SNORT. They showed that RACC identified multiple correlated alerts and those correlated alerts are associated with all steps in two DDoS attack scenarios. Khosravi et al. [19] evaluated their method on a bespoke cybersecurity dataset and showed that their method detected alerts which are correlated to hosts exposed to APT attacks. Furthermore, they showed that their method detected alerts that correspond to the long dwell time behavior associated with APT attacks. Zhang et al. [16] showed that their multistep attack detection model identified alerts that are associated with denial-of-service attacks, memory exceptions, abnormal data access, and system and hardware exceptions. Lajevardi et al. [15] identified correlations between operating system and network events and showed that those correlated events are associated with multiple violations of security policy.

Xiao *et al.* [10] evaluated their DDoS detection approach on three datasets which include traces from a trans-Pacific line, traces from a data center located in Dalian and KDD Cup 1999. They showed that their approach detected attacks on multiple network protocols such as HTTP and DNS. Furthermore, they showed that multiple types of attacks can be detected. Cheng et al. [25] evaluated their framework on the DARPA99 dataset and showed that their approach identified multiple alerts that are correlated to several attack scenarios. Vidal et al. [18] evaluated their strategy on real network traffic obtained from a data center operated by University Complutense of Madrid. They showed that their approach identified multiple alerts that are correlated to several categories of attacks including DoS, information leak, virus, trojan, spyware and unauthorized access. Diakhame et al. [33] evaluated their framework on the UNSW-NB15 dataset and showed that their framework identified several alerts that are associated with multiple cyberattack scenarios including DoS, worms, shellcode, fuzzers and backdoors. Kidmose et al. [20] evaluated their method on the CTU botnet dataset. They showed that their method identified several alerts that are associated with multiple incidents of botnet attacks. Subsequently, Kidmose et al. [30] evaluated their correlation and filtering approach on one private cybersecurity dataset and one public cybersecurity dataset. They showed that their approach

Table 12
We use the following acronyms to represent the six quality attributes in the reviewed SysLogCT approaches. S.R - Spurious Relationships, M.D.V - Missing Data Values, C.T - Correlation Threshold, B.&V. - Bias & Variance, D.A - Data Anomalies, E.M - Evaluation Metrics, F.P.R - False Positive Rate and F.N.R - False Negative Rate. We use the following acronyms to represent a domain. N.S - Not Specified, Adj. - Adjustable.

Tool	S.R	M.D.V	C.T	B.&V.	D.A	E.M		Dataset
						F.P.R	F.N.R	
Mahdavi et al. [14]	No	No	Fixed	N.S	N.S	No	No	DARPA2000, ASPA Co. 2019
Khosravi et al. [19]	No	No	Adj.	Yes	N.S	Yes	No	Custom dataset
Lajevardi et al. [15]	No	No	Adj.	N.S	N.S	No	No	Custom dataset
Zhang et al. [16]	No	No	Adj.	N.S	N.S	No	No	DARPA2000, China Grid
Xiao *et. al.* [10]	No	No	Adj.	Yes	N.S	No	No	KDD99, Real dataset Wide dataset
Cheng et al. [25]	No	No	Fixed	Yes	N.S	No	No	DARPA1999
Vidal et al. [18]	No	No	Adj.	Yes	N.S	Yes	Yes	UCM
Diakhame et al. [33]	No	No	Adj.	N.S	N.S	No	No	DEFCON CTF23
Kidmose et al. [20]	No	No	Fixed	Yes	N.S	No	No	CTU
Kidmose et al. [30]	No	No	Adj.	Yes	N.S	No	Yes	MCFP, CICIDS2017
Ghafir et al. [24]	No	No	Adj.	Yes	N.S	Yes	No	Custom dataset
Mishra et al. [22]	No	No	N.S	Yes	Yes	No	No	BoT–IoT
Haas et al. [27]	No	No	Fixed	N.S	N.S	No	No	DShield, Custom dataset
Grammatikis et al. [35]	No	No	Adj.	Yes	N.S	Yes	No	Custom dataset
Prasad et al. [13]	No	No	Fixed	Yes	Yes	Yes	No	UNSW NB15
Padmashree & Krishnamoorthi [23]	No	No	Fixed	Yes	Yes	Yes	No	BoT–IoT
Wei et al. [17]	No	No	Adj.	N.S	Yes	Yes	Yes	Simulation
Zhang et al. [21]	No	No	N.S	N.S	Yes	No	No	CERNET
Chuah et al. [26]	No	No	Fixed	N.S	Yes	Yes	No	NetFlow
Chawla et al. [32]	No	No	Fixed	N.S	No	No	No	FIFA1998, CAIDA, MIT, GENI
Gottwalt et al. [12]	No	No	Fixed	N.S	Yes	Yes	No	UNSW NB15
Ramaki et al. [11]	No	No	Adj.	N.S	N.S	No	No	DARPA2000
Revathi [31]	No	No	Adj.	N.S	Yes	No	No	Simulation
Yang et al. [34]	No	No	Adj.	N.S	N.S	Yes	No	Custom dataset
Chuah & Suri [28]	No	No	N.S	Yes	N.S	No	No	NetFlow
Amma & Subramanian [29]	No	No	Adj.	N.S	Yes	Yes	No	KDD99, NSL-KDD, UNSW NB15

identified sequences of alerts that are associated with multiple attack scenarios including DoS, DDoS, port scans and SQL injection.

Ghafir et al. [24] evaluated their system on a dataset that contains multiple APT attack scenarios and several alerts. They showed that their system identified multiple alerts that are associated with all four steps in several APT attack scenarios. Mishra et al. [22] showed that several classifiers trained on a small number of strongly correlated features achieved the highest accuracy in detecting DDoS attacks on the BoT–IoT dataset. Haas et al. [27] showed that network motif signatures can identify 96% of attacks and only 1% of the full dataset is required for identifying alerts that are associated with those attacks. Grammatikis et al. [35] evaluated their SIEM system on four real smart grid use cases and showed that their system can identify correlated cyberattacks on multiple smart grid application-layer protocols. Prasad et al. [13] showed that a small set of strongly correlated features in the UNSW-NB15 dataset can be used to train a highly accurate decision tree classifier and detect several attacks. Differently to Prasad et al. [13], Padmashree & Krishnamoorthi [23] showed that a DNN model trained with a small set of features selected by a decision-tree based Pearson correlation technique detected several attacks in the BoT–IoT dataset with a 99.2% accuracy.

Wei et al. [17] showed that their Spearman-Rank correlation based method detected several reflection DDoS attack cases with a low false negative of 0.18% and a low false positive of 0.1%. Zhang et al. [21] evaluated their Pearson correlation-based method on a dataset that contains CodeRed worm traces and showed that their method identified several anomalies that are correlated by time. Chuah et al. [26] showed that the Spearman-Rank correlation algorithm identified reflection attacks on the SSH and DNS servers on several dates. Subsequently, Chuah & Suri [28] showed that reflection attacks on the NTP servers and NetBIOS servers are not correlated by time. Chawla et al. [32] showed that their Pearson correlation-based DDoS detection framework distinguished DDoS attack traffic and flash events traffic. Amma & Subramanian [29] also developed a detection system that distinguished normal and DoS network traffic. Different to the system developed by Chawla et al. [32], Amma & Subramanian [29] used feature correlation maps. The detection system developed by Revathi [31] also distinguished malicious and legitimate network traffic. In contrast to the detection systems developed by Chawla et al. [32] and Amma & Subramanian [29], Revathi [31] used a flow and rank-based correlation algorithm to distinguish malicious and legitimate network traffic.

Gottwalt et al. [12] showed that the performance of anomaly detection (AD) systems that use multivariate correlation feature selection techniques consistently outperform AD systems that use Principal Component Analysis and Pearson correlation. Ramaki et al. [11] developed an alert correlation framework that integrated stream mining techniques and causal correlation analysis, and showed that their framework detected known and new attack scenarios with 95% accuracy. Yang et al. [34] developed an APT detection system that uses semantic analysis and causal correlation, and showed that their system reconstructed several APT attack scenarios and identified several alerts that are correlated to multiple APT attacks.

7.1.1. Observations

The SysLogCT approaches reviewed are effective in detecting several types of attacks with high accuracy. However, if: (a) the correlated features are due to the presence of one or more confounding features, or (b) the correlated network events are due to the presence of one or more confounding network events, then this can reduce the accuracy of a cyberattack detector or predictor [98].

7.1.2. Promising research directions

There is little work that identified spurious relationships between features or network events, which may lead to a wrong detection or prediction of a cyberattack. In statistics, identifying spurious relationships between variables is important for ruling out variables which do not represent a causal relationship [55]. A spurious relationship can be identified when the effect of one or more confounding

variables is controlled. Partial correlation measures the degree of association between two variables and controls the effect of one or more confounding variables [99] in a correlation model. Conversely, the coefficient-of-determination and Durbin–Watson's test can be used to identify spurious relationships in a regression model [100]. As such, researchers could investigate further statistical tests to identify spurious relationships in a correlation model or a regression model. While current SysLogCT approaches are highly effective in detecting or predicting cyberattacks, methods which can reduce the probability of making a wrong detection or prediction of a cyberattack could be a valuable addition to current SysLogCT approaches.

7.2. Missing data values

From Table 12, we observed that all the reviewed SysLogCT approaches did not determine their performance against missing system log-events or data loss. The approaches developed by Xiao *et al.* [10], Ramaki et al. [11], Gottwalt et al. [12], Prasad et al. [13], Mahdavi et al. [14], Lajevardi et al. [15], Zhang et al. [16], Wei et al. [17], Vidal et al. [18], Khosravi et al. [19], Kidmose et al. [20,30], Zhang et al. [21], Mishra et al. [22], Padmashree & Krishnamoorthi [23], Ghafir et al. [24], Cheng et al. [25], Chuah et al. [26,28], Haas et al. [27], Amma & Subramanian [29], Revathi [31], Chawla et al. [32], Diakhame et al. [33], Yang et al. [34] and Grammatikis et al. [35] assumed that the dataset contains all events associated with malicious and normal activities. 5

7.2.1. Observations

There are few studies that investigated the impact of missing network events or data loss on the ability of a SysLogCT approach to detect or predict a cyberattack.

7.2.2. Promising research directions

Researchers could evaluate the robustness of a SysLogCT approach against missing feature values or missing data samples. For example, generate DDoS attack network traffic and match the generated DDoS attack traffic to DDoS traces in the network logs to determine if there are missing values. Researchers could also investigate techniques for handling missing data values, for example, mean substitution, regression imputation or K-Nearest Neighbor imputation [96].

7.3. Correlation threshold value

From Table 12, we observed that the correlation threshold value in the reviewed SysLogCT approaches can be divided into three domains: (a) Adjustable, (b) Fixed, and (c) Not Specified. The reviewed SysLogCT approaches that implemented adjustable correlation thresholds are from Xiao et al. [10], Ramaki et al. [11], Lajevardi et al. [15], Wei et al. [17], Vidal et al. [18], Khosravi et al. [19], Ghafir et al. [24], Amma & Subramanian [29], Revathi [31], Diakhame et al. [33], Yang et al. [34], Grammatikis et al. [35], Zhang et al. [16] and Kidmose et al. [30]. The reviewed SysLogCT approaches that implemented a fixed correlation threshold value are from Gottwalt et al. [12], Prasad et al. [13], Mahdavi et al. [14], Kidmose et al. [20], Padmashree & Krishnamoorthi [23], Cheng et al. [25], Chuah et al. [26], Haas et al. [27] and Chawla et al. [32]. The reviewed SysLogCT approaches that did not indicate a correlation threshold value or a correlation threshold criterion are from Zhang et al. [21], Mishra et al. [22] and Chuah & Suri [28].

Xiao et al. [10] used the parameter k in their CKNN algorithm to evaluate its performance. The parameter k specifies the number of network flows to check to determine if a network flow belongs to an attack class, and a range of different k values are used. Ramaki et al. [11] used two separate thresholds to determine if a sequence of events called an episode is a malicious episode. Wei et al. [17] also used two separate thresholds to determine if two network flows are malicious or not. Similar to Wei et al. [17], Revathi [31] defined two separate threshold values to determine if two network flows are malicious or not. Ghafir et al. [24] used three separate values to determine if alerts are correlated. Amma & Subramanian [29] defined two separate threshold values; one that selected the strongly correlated features and the second one that classified DoS network traffic. Yang et al. [34] obtained a range of threshold values by setting it to the Average Causal Effect measure for the top 5% of alerts. Unlike Xiao et al. [10], Ramaki et al. [11], Wei et al. [17], Revathi [31], Ghafir et al. [24], Amma & Subramanian [29] and Yang et al. [34], Lajevardi et al. [15] defined a set of security rules and used those rules to determine if security events belonged to an APT attack. Vidal et al. [18] also defined a set of rules and used those rules to identify suspicious network packets. Khosravi et al. [19] also defined a set of rules. Unlike Lajevardi et al. [15] and Vidal et al. [18], Khosravi et al. [19] used those rules to determine if a causal relationship exists between alerts. Diakhame et al. [33] used the number of NER keywords to filter alerts that are not important. Alerts that do not contain any NER keywords contained in other alerts are removed. Grammatikis et al. [35] defined a set of rules to determine a severity level for each security event. Zhang et al. [16] defined several rules to identify negative causal correlations between alerts in an attack chain. Kidmose et al. [30] defined two separate similarity functions to identify groups of correlated alerts.

Gottwalt et al. [12], Prasad et al. [13], Kidmose et al. [20] and Chuah et al. [26] defined a fixed correlation threshold of 0.8, 0.9, 0.5 and 0.8 respectively. Gottwalt et al. [12] and Prasad et al. [13] used the correlation threshold value to identify and remove redundant features. Unlike Gottwalt et al. [12] and Prasad et al. [13], Kidmose et al. [20] and Chuah et al. [26] used the correlation threshold value to identify strongly positive correlated network events. Mahdavi et al. [14] defined a single distance criterion to identify alerts that are correlated to other alerts in the same network. Chawla et al. [32] also specified a fixed correlation threshold to determine if two network flows are similar. Furthermore, Cheng et al. [25] and Padmashree & Krishnamoorthi [23] also specified fixed correlation thresholds. Cheng et al. [25] used the correlation threshold to identify similar alerts. Differently to Cheng et al. [25], Padmashree & Krishnamoorthi [23] used the correlation threshold to rank the correlated features. Haas et al. [27] defined a minimum threshold to identify similar attack scenarios.

7.3.1. Observations

The reviewed SysLogCT approaches implemented different techniques that obtained a range of correlation threshold values or used different predefined correlation threshold values. Consequently, it is challenging to ascertain the impact the correlation threshold values have on the ability of a SysLogCT approach to detect or predict cyberattacks, especially when different techniques are implemented in these SysLogCT approaches.

7.3.2. Promising research directions

There is little work that assessed the correlation threshold value's impact on the ability of a SysLogCT approach to detect or predict cyberattacks. Thus, researchers could conduct an empirical study on how the correlation threshold value can be optimized to improve the accuracy of a SysLogCT approach in detecting or predicting cyberattacks.

7.4. Bias and variance

From Table 12, we observed that the reviewed SysLogCT approaches can be divided into two domains: (a) SysLogCT approaches that implemented ML models, and (b) SysLogCT approaches that did not implement ML models. We focused on the reviewed SysLogCT approaches that implemented ML models. These SysLogCT approaches were developed by Xiao *et al.* [10], Cheng et al. [25], Vidal et al. [18], Kidmose et al. [20,30], Ghafir et al. [24], Mishra et al. [22], Grammatikis

et al. [35], Prasad et al. [13], Padmashree & Krishnamoorthi [23] and Chuah & Suri [28].

Xiao *et al.* [10] divided the training dataset into several subsets and used those subsets to train their CKNN model, and a separate test dataset is used to evaluate the performance of their CKNN model on unseen traffic flows. Similar to Xiao *et al.* [10], Grammatikis et al. [35] used many training and test datasets to train and test various ML and deep learning models. Ghafir et al. [24], Prasad et al. [13] and Kidmose et al. [30] also used k-fold cross-validation to evaluate the ML models implemented in their respective frameworks. In contrast to Xiao *et al.* [10], R-Grammatikis et al. [35], Ghafir et al. [24], Prasad et al. [13] and Kidmose et al. [30], Cheng et al. [25] used a range of values in the bias and weight parameters in every layer in the GCN to prevent overfitting their model to the training data. Mishra et al. [22] applied $L1$-regularization and $L2$-regularization in the XGBoost algorithm, and assigned weights to weak models in the AdaBoost algorithm. Padmashree & Krishnamoorthi [23] controlled the amount of bias and variance in their DT-PCRFE model by specifying weights for the penalty parameter in the decision tree classifier and updating the weights in the DNN model. Chuah & Suri [28] specified a range of values for the penalty parameter in the Ridge, LASSO and Elastic Net regression models to prevent overfitting these regression models on the training data. Vidal et al. [18] used 80% of data to train their alert correlation framework and used 20% of the data to test their framework. Kidmose et al. [20] divided the data into one training set, one validation set and one testing set.

7.4.1. Observations

The majority of the reviewed SysLogCT approaches that implemented ML models are robust against bias and variance. These SysLogCT approaches used k-fold cross-validation or regularization techniques that prevented overfitting the ML model on the training data. The SysLogCT approaches developed by Vidal et al. [18] used one training dataset and one testing dataset, and Kidmose et al. [20] used one training dataset, one testing dataset and one validation dataset; these two SysLogCT approaches are likely robust against bias and variance. However, if the data is incomplete or faulty, it may cause the SysLogCT approach to overfit or underfit the ML model on the dataset.

7.4.2. Promising research directions

There is little work that investigated the impact of data quality on the performance of a SysLogCT approach. Thus, researchers could develop further experiments to evaluate the ML model on good and poor quality data. While current SysLogCT approaches are robust or likely robust against bias and variance, methods that are robust against bias and variance in the presence of poor quality data could be a valuable addition to current SysLogCT approaches.

7.5. Data anomalies

From Table 12, we observed that Mishra et al. [22], Prasad et al. [13], Padmashree & Krishnamoorthi [23], Wei et al. [17], Zhang et al. [21], Chuah et al. [26], Chawla et al. [32], Gottwalt et al. [12], Revathi [31] and Amma & Subramanian [29] implemented statistical methods in their approach.

Mishra et al. [22] implemented two feature selection methods based on the Pearson correlation and Kendall Tau correlation algorithms and used these feature selectors to select features for training the Decision tree, XGBoost, AdaBoost and Random forest classifiers. Prasad et al. [13] rescaled all data values to range between 0 and 1 and used the Pearson correlation algorithm to identify and remove features that were strongly correlated. Padmashree & Krishnamoorthi [23] also rescaled all data values to range between 0 and 1, and used the Pearson correlation algorithm to identify the important features. Wei et al. [17] used the Spearman-Rank correlation algorithm to identify alerts that were strongly correlated. Chuah et al. [26] used the Pearson correlation and Spearman-Rank correlation algorithms to identify dates of network attacks. Chawla et al. [32] used the Pearson correlation algorithm to distinguish DDoS events from flash-based events. Gottwalt et al. [12] implemented the symmetric tau correlation algorithm []symmetric-tau-1991 and used it to identify features that were strongly correlated. Revathi [31] used the Spearman-Rank correlation algorithm to identify strongly correlated network flows. Amma & Subramanian [29] rescaled all data values to range between a specified minimum and maximum value and then applied the Pearson correlation algorithm on the rescaled data to select the important features. Zhang et al. [21] applied the Pearson correlation algorithm on the percentage change value between consecutive traffic flows to identify traffic flows associated with the Code Red worm.

7.5.1. Observations

Robust statistical correlation methods have been used to select important features to train a ML classifier or to identify traffic flows that are correlated to a cyberattack. However, there are very few works that investigated the impact of data anomalies on the accuracy of a SysLogCT approach in detecting or predicting cyberattacks.

7.5.2. Promising research directions

Researchers could investigate novel correlation methods that are robust in the presence of data anomalies, for example, using a regression model that explicitly detects anomalies in the data [101] rather than using Spearman-Rank correlation or Kendall Tau Correlation.

7.6. Evaluation metrics

From Table 12, we observed that: (a) 2 out of the 26 reviewed SysLogCT approaches were evaluated *wrt.* the false positive rate and false negative rate, (b) 9 out of the 26 reviewed SysLogCT approaches were evaluated *wrt.* the false positive rate only, (c) 1 out of the 26 reviewed SysLogCT approaches was evaluated *wrt.* the false negative rate only, and (d) 14 out of the 26 reviewed SysLogCT approaches were not evaluated *wrt.* the false positive rate or false negative rate.

Wei et al. [17] used two thresholds to determine if two network flows are malicious or not. They simulated 200 different attack cases and showed that when threshold values of -0.15 and 0.3 are used, their method achieved a low FPR and FNR of 0.1% and 0.18% respectively. Vidal et al. [18] showed that filtering alerts with low and very low severity resulted in an 95.7% improvement in the false positive rate but increased the false negative rate by 7.3%.

Khosravi et al. [19] used the recall, precision, accuracy, F1-score and FPR metrics to evaluate their approach and their results showed that their approach achieved a low FPR of 13.79%. R-Grammatikis et al. [35] used the accuracy, true positive rate, F1-score and FPR metrics to evaluate *SPEAR SIEM* and showed that their detection model achieved low FPR values. Prasad et al. [13] used the accuracy, detection rate and FPR metrics to evaluate their feature selection method. They showed that when a Decision tree classifier is trained using features from their feature selection method, the Decision tree classifier achieved a low FPR of 12%. Padmashree & Krishnamoorthi [23] used the accuracy, precision, recall, F1-score, Area Under the Curve (AUC) and FPR metrics to evaluate their BoT–IoT attack detection model. Their results showed that their attack detection model achieved a low FPR of 0.001%. Chuah et al. [26] used the Bonferroni Correction to account for inflation in false positives and showed that it is highly unlikely that two strongly positive correlated dates associated with a network attack would be false. Gottwalt et al. [12] used the accuracy, detection rate and FPR metrics to evaluate *CorrCorr*. Their results showed that *CorrCorr* achieved the lowest FPR of 1.26% compared to four existing techniques. Yang et al. [34] used the accuracy, recall and FPR metrics to evaluate *Poirot* on multiple security logs that contained false alerts, and showed that the FPR is low when the number of false alerts ranged from 0 to 15000. Amma & Subramanian [29] used the

accuracy, true positive rate, error rate and FPR metrics to evaluate their attack detection model, and showed that their attack detection model achieved a low FPR.

Kidmose et al. [30] defined a new metric called *Incident Miss Rate* (IMR) that determines the probability that a random security incident will be missed. They evaluated the performance of four different implementations of their approach using the IMR metric. Their results showed that all the four implementations achieved low mean IMR values that ranged from $7.26e^{-17}$ to $6.12e^{-2}$.

7.6.1. Observations

All reviewed SysLogCT approaches were evaluated using one or more evaluation metrics. While these SysLogCT approaches are effective in detecting or predicting a cyberattack, there are few works that investigated the impact of false positives and false negatives on the accuracy of a SysLogCT approach to detect or predict a cyberattack.

7.6.2. Promising research directions

Researchers could apply new evaluation metrics which account for true positive, true negative, false positive and false negative, for example, using Matthew's Correlation Coefficient that accounts for true positive, true negative, false positive and false negative [102] to evaluate the performance of a binary classifier in addition to the precision, recall, accuracy, F1-score, FPR and FNR metrics.

8. Threats to validity

In a quantitative or qualitative study, the purpose of a threat to validity section is to acknowledge factors that might skew the data collected or have an undue influence on the research presented. We identified three validity threats: (a) threats to construct validity, (b) threats to internal validity, and (c) threats to external validity.

Construct validity is concerned with how well a set of attributes represents or reflects observations that are not directly measurable. We have developed a qualitative quality model that covers six important quality attributes of SysLogCT approaches. The quality attributes of spurious relationships, missing data, correlation threshold values, bias and variance, anomalies in the data, and evaluation metrics were identified from a thorough and detailed review of 62 representative articles that we obtained through our systematic literature review. As these 62 articles are representative of SysLogCT approaches, we believe that there is no threat to the accuracy of our results in Section 7. However, some articles may be missed. To address this issue, we followed established guidelines by Kitchenham et al. [45–47] on how to perform a systematic literature review in software engineering.

Internal validity is concerned with whether there is sufficient evidence to support the results presented in this paper. Factors that might influence the results include the selection of articles and the choice of quality attributes of these SysLogCT approaches. Regarding selection of papers, we may have missed relevant papers on SysLogCT approaches. We have addressed this issue by following established procedures for conducting a systematic literature review [46]. We used Google Scholar as the main search engine because it contains indexes to multiple articles and these articles are published in several digital libraries such as IEEE, ACM, Springer and Elsevier. To mitigate the risk of missing relevant papers, we performed forward and backward snowballing. Our analysis of the quality attributes of SysLogCT approaches are based on the approach and results presented in the selected papers. SysLogCT approaches for reliability were not reviewed because these works are beyond the scope of this paper. Furthermore, we did not implement the SysLogCT approach or check the source code because it requires significant resources beyond the scope of this paper.

External validity is concerned with how the results can be generalized. The conclusions presented in this paper are based on 26 unique SysLogCT approaches and may not generalize to all SysLogCT approaches. However, the 26 SysLogCT approaches (see Table 12) were evaluated on large cybersecurity datasets. As such, we believe that the outcomes presented in Section 7 are representative of most SysLogCT approaches. Regardless, we have shown that our quality model can be used to evaluate any SysLogCT approach that detects or predicts cyberattacks.

9. Conclusion

A systematic literature review of SysLogCT approaches is presented. We obtained 4,599 initial articles and selected 62 representative articles. We reviewed these 62 articles in detail and identified important quality attributes associated with the SysLogCT approach described in the article. We developed a quality model that consisted of six quality attributes including spurious relationships, missing data or network events, correlation threshold value, bias and variance, anomalies in the data and evaluation metrics, and compared and evaluated 26 unique SysLogCT approaches using our quality model. From our results, we learned that not one SysLogCT approach met all the six quality attributes.

The security analyst can use the results in Section 7 to identify a SysLogCT approach that is relevant to their specific use case. Cybersecurity researchers can compare and evaluate SysLogCT approaches using our quality model; they can also use our recommendations to develop a new SysLogCT approach or enhance an existing SysLogCT approach. We made six recommendations for future research. Our recommendations are: (a) develop novel approaches that can detect spurious relationships so that the risk of making a wrong detection or prediction of a cyberattack is reduced, (b) develop novel approaches that are robust against missing network events, (c) investigate how the correlation threshold values can be fine-tuned to improve detection or prediction accuracy, (d) study the impact that quality of data can have on the performance of a SysLogCT approach, (e) develop novel approaches that are robust against anomalies in the data, and (f) apply new evaluation metrics that provide robust evaluation of the performance of a SysLogCT approach.

CRediT authorship contribution statement

Edward Chuah: Writing – original draft, Methodology, Data curation, Conceptualization. **Harsha Kalutarage:** Writing – review & editing, Methodology. **Kasim Tasdemir:** Writing – review & editing, Methodology. **Atnafu Abrham:** Writing – review & editing. **Carsten Maple:** Writing – review & editing.

Ethical approval

Not applicable.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive feedback, which helped improve our manuscript significantly.

Funding

No funding was received to assist with the preparation of this manuscript.

Data availability

Data will be made available on request.

References

[1] Nour B, Pourzandi M, Debbabi M. A survey on threat hunting in enterprise networks. IEEE Commun Surv Tutor 2023;25(4):2299–324. http://dx.doi.org/10.1109/COMST.2023.3299519.

[2] Ferreira L, Silva DC, Itzazelaia MU. Recommender systems in cybersecurity. Knowl Inf Syst 2023;65(12):5523–59. http://dx.doi.org/10.1007/s10115-023-01906-6.

[3] Hofstede R, Čeleda P, Trammell B, Drago I, Sadre R, Sperotto A, et al. Flow monitoring explained: From packet capture to data analysis with NetFlow and IPFIX. IEEE Commun Surv Tutor 2014;16(4):2037–64. http://dx.doi.org/10.1109/COMST.2014.2321898.

[4] Lin H, Yan Z, Chen Y, Zhang L. A survey on network security-related data collection technologies. IEEE Access 2018;6:18345–65. http://dx.doi.org/10.1109/ACCESS.2018.2817921.

[5] Yusof R, Selamat SR, Sahib S, Mas'ud MZ, Abdollah MF. Enhanced alert correlation framework for heterogeneous log. In: International conference on informatics engineering and information science. Springer; 2011, http://dx.doi.org/10.1007/978-3-642-25327-0_10.

[6] Mora-Gimeno FJ, Mora-Mora H, Volckaert B, Atrey A. Intrusion detection system based on integrated system calls graph and neural networks. IEEE Access 2021;9:9822–33. http://dx.doi.org/10.1109/ACCESS.2021.3049249.

[7] Fernandes G, Carvalho LF, Rodrigues JJ, Proença ML. Network anomaly detection using IP flows with principal component analysis and ant colony optimization. J Netw Comput Appl 2016;64:1–11. http://dx.doi.org/10.1016/j.jnca.2015.11.024.

[8] da Silva L, Coury D. Network traffic prediction for detecting DDoS attacks in IEC 61850 communication networks. Comput Electr Eng 2020;87:1–14. http://dx.doi.org/10.1016/j.compeleceng.2020.106793.

[9] Shang Y. False positive and false negative effects on network attacks. J Stat Phys 2018;170:141–64. http://dx.doi.org/10.1007/s10955-017-1923-7.

[10] Xiao P, Qu W, Qi H, Li Z. Detecting DDoS attacks against data center with correlation analysis. Comput Commun 2015;67:66–74. http://dx.doi.org/10.1016/j.comcom.2015.06.012.

[11] Ramaki AA, Amini M, Ebrahimi Atani R. RTECA: Real time episode correlation algorithm for multi-step attack scenarios detection. Comput Secur 2015;49:206–19. http://dx.doi.org/10.1016/j.cose.2014.10.006.

[12] Gottwalt F, Chang E, Dillon T. CorrCorr: A feature selection method for multivariate correlation network anomaly detection techniques. Comput Secur 2019;83:234–45. http://dx.doi.org/10.1016/j.cose.2019.02.008.

[13] Prasad M, Gupta RK, Tripathi S. A multi-level correlation-based feature selection for intrusion detection. Arab J Sci Eng 2022;47(8):10719–29. http://dx.doi.org/10.1007/s13369-022-06760-2.

[14] Mahdavi E, Fanian A, Amini F. A real-time alert correlation method based on code-books for intrusion detection systems. Comput Secur 2020;89:101661. http://dx.doi.org/10.1016/j.cose.2019.101661.

[15] Lajevardi AM, Amini M. A semantic-based correlation approach for detecting hybrid and low-level APTs. Future Gener Comput Syst 2019;96:64–88. http://dx.doi.org/10.1016/j.future.2019.01.056.

[16] Zhang H, Jin X, Li Y, Jiang Z, Liang Y, Jin Z, et al. A multi-step attack detection model based on alerts of smart grid monitoring system. IEEE Access 2019;8:1031–47. http://dx.doi.org/10.1109/ACCESS.2019.2961517.

[17] Wei W, Chen F, Xia Y, Jin G. A rank correlation based detection against distributed reflection DoS attacks. IEEE Commun Lett 2013;17(1):173–5. http://dx.doi.org/10.1109/LCOMM.2012.121912.122257.

[18] Vidal JM, Orozco ALS, Villalba LJG. Alert correlation framework for malware detection by anomaly-based packet payload analysis. J Netw Comput Appl 2017;97:11–22. http://dx.doi.org/10.1016/j.jnca.2017.08.010.

[19] Khosravi M, Ladani BT. Alerts correlation and causal analysis for APT based cyberattack detection. IEEE Access 2020;8:162642–56. http://dx.doi.org/10.1109/ACCESS.2020.3021499.

[20] Kidmose E, Stevanovic M, Pedersen JM. Correlating intrusion detection alerts on bot malware infections using neural network. In: Proceedings of the international conference on cyber security and protection of digital services. IEEE; 2016, p. 1–8. http://dx.doi.org/10.1109/CyberSecPODS.2016.7502344.

[21] Zhang Q, Wang J, Li X. Correlation based analysis of spreading CodeRed worms. In: Proceedings of the international conference on intelligent control and information processing. IEEE; 2010, p. 458–62. http://dx.doi.org/10.1109/ICICIP.2010.5564187.

[22] Mishra A, Soni J, Jaiswal P, Chavan S, Kolhe K, Mante J. Critical Feature Selection (CFS) techniques for early DDoS attack detection in the internet of things environment. In: Proceedings of the 7th international conference on computing, communication, control and automation. IEEE; 2023, p. 1–6. http://dx.doi.org/10.1109/ICCUBEA58933.2023.10392251.

[23] Padmashree A, Krishnamoorthi M. Decision tree with Pearson correlation-based recursive feature elimination model for attack detection in IoT environment. Inf Technol Control 2022;51(4):771–85. http://dx.doi.org/10.5755/j01.itc.51.4.31818.

[24] Ghafir I, Hammoudeh M, Prenosil V, Han L, Hegarty R, Rabie K, et al. Detection of advanced persistent threat using machine-learning correlation analysis. Future Gener Comput Syst 2018;89:349–59. http://dx.doi.org/10.1016/j.future.2018.06.055.

[25] Cheng Q, Wu C, Zhou S. Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. IEEE Commun Lett 2021;25(5):1564–7. http://dx.doi.org/10.1109/LCOMM.2020.3048995.

[26] Chuah E, Suri N, Jhumka A, Alt S. Challenges in identifying network attacks using netflow data. In: Proceedings of IEEE international symposium on network computing and applications. 2021, http://dx.doi.org/10.1109/NCA53618.2021.9685305.

[27] Haas S, Wilkens F, Fischer M. Efficient attack correlation and identification of attack scenarios based on network-motifs. In: Proceedings of the IEEE 38th international performance computing and communications conference. IEEE; 2019, p. 1–11. http://dx.doi.org/10.1109/IPCCC47392.2019.8958734.

[28] Chuah E, Suri N. An empirical study of reflection attacks using NetFlow data. Cybersecurity 2024;7(13):1–22. http://dx.doi.org/10.1186/s42400-023-00203-7.

[29] Amma NGB, Subramanian S. Feature correlation map based statistical approach for denial of service attacks detection. In: Proceedings of the IEEE 5th international conference on computing engineering and design. IEEE; 2019, p. 1–6. http://dx.doi.org/10.1109/ICCED46541.2019.9161080.

[30] Kidmose E, Stevanovic M, Brandbyge S, Pedersen JM. Featureless discovery of correlated and false intrusion alerts. IEEE Access 2020;8:108748–65. http://dx.doi.org/10.1109/ACCESS.2020.3001374.

[31] Revathi P. Flow and rank correlation based detection against distributed reflection denial of service attack. In: Proceedings of the IEEE international conference on recent trends in information technology. IEEE; 2014, p. 1–6. http://dx.doi.org/10.1109/ICRTIT.2014.6996117.

[32] Chawla S, Sachdeva M, Behal S. Discrimination of DDoS attacks and flash events using Pearson's product moment correlation method. Int J Comput Sci Inf Security (IJCSIS) 2016;14(10):382–9, Available at https://sites.google.com/site/ijcsis/all-volumes-issues/vol-14-no-10-oct-2016.

[33] Diakhame ML, Diallo C, Mejri M. MCM-CASR: Novel alert correlation framework for cyberattack scenario reconstruction based on NLP, NER, and semantic similarity. In: Proceedings of the IEEE 7th cyber security in networking conference. IEEE; 2023, p. 27–31. http://dx.doi.org/10.1109/CSNet59123.2023.10339751.

[34] Yang J, Zhang Q, Jiang X, Chen S, Yang F. Poirot: Causal correlation aided semantic analysis for advanced persistent threat detection. IEEE Trans Dependable Secur Comput 2021;19(5):3546–63. http://dx.doi.org/10.1109/TDSC.2021.3101649.

[35] Radoglou-Grammatikis P, Sarigiannidis P, Iturbe E, Rios E, Martinez S, Sarigiannidis A, et al. Spear SIEM: A security information and event management system for the smart grid. Comput Netw 2021;193:108008. http://dx.doi.org/10.1016/j.comnet.2021.108008.

[36] Kotenko I, Gaifulina D, Zelichenok I. Systematic literature review of security event correlation methods. IEEE Access 2022;10:43387–420. http://dx.doi.org/10.1109/ACCESS.2022.3168976.

[37] Levshun D, Kotenko I. A survey on artificial intelligence techniques for security event correlation: models, challenges, and opportunities. Artif Intell Rev 2023;56:8547–90. http://dx.doi.org/10.1007/s10462-022-10381-4.

[38] López Velásquez JM, Martínez Monterrubio SM, Sánchez Crespo LE, Garcia Rosado D. Systematic review of SIEM technology: SIEM-SC birth. Int J Inf Secur 2023;22(3):691–711. http://dx.doi.org/10.1007/s10207-022-00657-9.

[39] de Neira AB, Kantarci B, Nogueira M. Distributed denial of service attack prediction: Challenges, open issues and opportunities. Comput Netw 2023;222:109553. http://dx.doi.org/10.1016/j.comnet.2022.109553.

[40] Khraisat A, Gondal I, Vamplew P, Kamruzzaman J. Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity 2019;2:20. http://dx.doi.org/10.1186/s42400-019-0038-7.

[41] Chou D, Jiang M. A survey on data-driven network intrusion detection. ACM Comput Surv 2021;54(9). http://dx.doi.org/10.1145/3472753.

[42] Gupta N, Jindal V, Bedi P. A survey on intrusion detection and prevention systems. SN Comput Sci 2023;4(5). http://dx.doi.org/10.1007/s42979-023-01926-7.

[43] El-Masri D, Petrillo F, Guéhéneuc Y-G, Hamou-Lhadj A, Bouziane A. A systematic literature review on automated log abstraction techniques. Inf Softw Technol 2020;122:106276. http://dx.doi.org/10.1016/j.infsof.2020.106276.

[44] Chuah E, Jhumka A, Malek M, Suri N. A survey of log-correlation tools for failure diagnosis and prediction in cluster systems. IEEE Access 2022;10:133487–503. http://dx.doi.org/10.1109/ACCESS.2022.3231454.

[45] Kitchenham B, Dyba T, Jorgensen M. Evidence-based software engineering. In: Proceedings of IEEE 26th international conference on software engineering. 2004, p. 273–81. http://dx.doi.org/10.1109/ICSE.2004.1317449.

[46] Kitchenham B, Charters S. Guidelines for performing systematic literature reviews in software engineering. Tech. rep., Keele University, UK; 2007, Available at https://tinyurl.com/yns656jf.

[47] Kitchenham B, Pearl Brereton O, Budgen D, Turner M, Bailey J, Linkman S. Systematic literature reviews in software engineering – A systematic literature review. Inf Softw Technol 2009;51(1):7–15. http://dx.doi.org/10.1016/j.infsof.2008.09.009.

[48] Vaarandi R, Podiņš K. Network IDS alert classification with frequent itemset mining and data clustering. In: Proceedings of the IEEE international conference on network and service management. 2010, p. 451–6. http://dx.doi.org/10.1109/CNSM.2010.5691262.

[49] Turcotte MJM, Kent AD, Hash C. Unified host and network data set. In: Data science for cyber-security. World Scientific; 2018, p. 1–22. http://dx.doi.org/10.1142/9781786345646_001.

[50] Los Alamos National Laboratories, USA. Unified Host and network data set. 2017, Available at https://csr.lanl.gov/data/2017.

[51] IEEE. IEEE Std 1003.1-2001 standard for information technology — Portable Operating System Interface (POSIX) base definitions, issue 6. USA: IEEE Standards; 2001, p. xliv + 448. http://dx.doi.org/10.1109/IEEESTD.2001.93364.

[52] Tan P-N, Steinbach M, Kumar V. Introduction to data mining. USA: Addison-Wesley; 2006.

[53] Agresti A, Franklin C. Statistics: The art and science of learning from data. USA: Prentice Hall; 2009.

[54] Cheng Q, Wu C, Zhou S. Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. IEEE Commun Lett 2021;25(5):1564–7. http://dx.doi.org/10.1109/LCOMM.2020.3048995.

[55] Simon HA. Spurious correlation: A causal interpretation. J Amer Statist Assoc 1954;49(267):467–79. http://dx.doi.org/10.2307/2281124.

[56] Emmanuel T, Maupong T, Mpoeleng D, Semong T, Mphago B, Tabona O. A survey on missing data in machine learning. J Big Data 2021;8:1–37. http://dx.doi.org/10.1186/s40537-021-00516-9.

[57] Piva A, Barni M, Bartolini F, Cappellini V. Threshold selection for correlation-based watermark detection. In: Proceedings of COST workshop on intelligent communications. 1998, p. 67–72, Available at https://cir.nii.ac.jp/crid/1570854175308613760.

[58] Schroeder LD, Sjoquist DL, Stephan PE. In: Understanding regression analysis: An introductory guide, vol. 57, USA: Sage Publications; 2016, http://dx.doi.org/10.4135/9781506361628.

[59] Holter JC, Stallrich JW. Tuning parameter selection for penalized estimation via R2. Comput Statist Data Anal 2023;183:107729. http://dx.doi.org/10.1016/j.csda.2023.107729.

[60] Kipoong Kim JK, Sun H. An empirical threshold of selection probability for analysis of high-dimensional correlated data. J Stat Comput Simul 2020;90(9):1606–17. http://dx.doi.org/10.1080/00949655.2020.1739286.

[61] Kim Y, Kim T-H, Ergün T. The instability of the Pearson correlation coefficient in the presence of coincidental outliers. Financ Res Lett 2015;13:243–57. http://dx.doi.org/10.1016/j.frl.2014.12.005.

[62] Filzmoser P, Nordhausen K. Robust linear regression for high-dimensional data: An overview. WIREs Comput Stat 2021;13(4):e1524. http://dx.doi.org/10.1002/wics.1524.

[63] Aggarwal CC. Outlier analysis. 2nd ed. Cham: Springer; 2016.

[64] Raymaekers J, Rousseeuw PJ. Fast robust correlation for high-dimensional data. Technometrics 2021;63(2):184–98. http://dx.doi.org/10.1080/00401706.2019.1677270.

[65] Yuen K-V, Ortiz GA. Outlier detection and robust regression for correlated data. Comput Methods Appl Mech Engrg 2017;313:632–46. http://dx.doi.org/10.1016/j.cma.2016.10.004.

[66] Negi CS, Kumari N, Kumar P, Sinha SK. An approach for alert correlation using ArcSight SIEM and open source NIDS. In: Nath V, Mandal JK, editors. Proceeding of fifth international conference on microelectronics, computing and communication systems. Singapore: Springer Singapore; 2021, p. 29–40. http://dx.doi.org/10.1007/978-981-16-0275-7_3.

[67] Brandao A, Georgieva P. Automatic log analysis to prevent cyberattacks. In: Sgurev V, Jotsov V, Kacprzyk J, editors. Advances in intelligent systems research and innovation. Cham: Springer International Publishing; 2022, p. 315–39. http://dx.doi.org/10.1007/978-3-030-78124-8_14.

[68] Li T, Ma J, Pei Q, Shen Y, Lin C, Ma S, et al. AClog: Attack chain construction based on log correlation. In: Proceedings of the IEEE global communications conference. 2019, p. 1–6. http://dx.doi.org/10.1109/GLOBECOM38437.2019.9013518.

[69] Hunt R, Hill S. Using security logs to identify and manage user behaviour to enhance information security. In: Proceedings of the 14th European conference on cyber warfare and security. 2015, p. 111, Available at https://tinyurl.com/36tnsas3.

[70] Xiao P, Qu W, Qi H, Li Z. Detecting DDoS attacks against data center with correlation analysis. Comput Commun 2015;67:66–74. http://dx.doi.org/10.1016/j.comcom.2015.06.012.

[71] Hall MA. Correlation-based feature selection for machine learning [Ph.D. thesis], The University of Waikato; 1999.

[72] Sarker IH. CyberLearning: Effectiveness analysis of machine learning security modeling to detect cyber-anomalies and multi-attacks. Internet Things 2021;14:100393. http://dx.doi.org/10.1016/j.iot.2021.100393.

[73] Heryanto A, Stiawan D, Bin Idris MY, Bahari MR, Hafizin AA, Budiarto R. Cyberattack feature selection using correlation-based feature selection method in an intrusion detection system. In: Proceedings of the international conference on electrical engineering, computer science and informatics. 2022, p. 79–85. http://dx.doi.org/10.23919/EECSI56542.2022.9946449.

[74] Hagberg A, Lemons N, Kent A, Neil J. Connected components and credential hopping in authentication graphs. In: Proceedings of the tenth international conference on signal-image technology and internet-based systems. 2014, p. 416–23. http://dx.doi.org/10.1109/SITIS.2014.95.

[75] Kent AD. Cyber security data sources for dynamic network research. In: Dynamic networks and cyber-security. p. 37–65. http://dx.doi.org/10.1142/9781786340757_0002.

[76] Beaver JM, Borges-Hink RC, Buckner MA. An evaluation of machine learning methods to detect malicious SCADA communications. In: Proceedings of the 12th IEEE international conference on machine learning and applications, vol. 2, 2013, p. 54–9. http://dx.doi.org/10.1109/ICMLA.2013.105.

[77] Morris T. Industrial Control System (ICS) cyberattack datasets. 2015, Available at https://sites.google.com/a/uah.edu/tommy-morris-uah/ics-data-sets.

[78] Canadian Institute for Cybersecurity. Datasets, Available at https://www.unb.ca/cic/datasets/index.html.

[79] Moustafa N, Slay J. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: Proceedings of the military communications and information systems conference. 2015, p. 1–6. http://dx.doi.org/10.1109/MilCIS.2015.7348942.

[80] Koroniotis N, Moustafa N, Sitnikova E, Turnbull B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. Future Gener Comput Syst 2019;100:779–96. http://dx.doi.org/10.1016/j.future.2019.05.041.

[81] Alsaedi A, Moustafa N, Tari Z, Mahmood A, Anwar A. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. IEEE Access 2020;8:165130–50. http://dx.doi.org/10.1109/ACCESS.2020.3022862.

[82] Dr Nour Moustafa. UNSW IoT-BoT datasets. 2021, Available at https://research.unsw.edu.au/projects/bot-iot-dataset.

[83] Rajapaksha S, Kalutarage H, Madzudzo G, Petrovski A, Al-Kadri M. CAN-MIRGU: A comprehensive CAN bus attack dataset from moving vehicles for intrusion detection system evaluation. In: Proceedings of the symposium on vehicle security and privacy in NDSS 2024. 2024, Available at https://www.ndss-symposium.org/ndss-paper/auto-draft-482/.

[84] Zhang J, Xiang Y, Wang Y, Zhou W, Xiang Y, Guan Y. Network traffic classification using correlation information. IEEE Trans Parallel Distrib Syst 2013;24(1):104–17. http://dx.doi.org/10.1109/TPDS.2012.98.

[85] Pivarníková M, Sokol P, Bajtoš T. Early-stage detection of cyberattacks. Information 2020;11(12). http://dx.doi.org/10.3390/info11120560.

[86] Devlin J, Chang M-W, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Burstein J, Doran C, Solorio T, editors. Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). Minneapolis, Minnesota: Association for Computational Linguistics; 2019, p. 4171–86. http://dx.doi.org/10.18653/v1/N19-1423.

[87] Milo R, Shen-Orr S, Itzkovitz S, Kashtan N, Chklovskii D, Alon U. Network Motifs: Simple building blocks of complex networks. Science 2002;298(5594):824–7. http://dx.doi.org/10.1126/science.298.5594.824.

[88] University of Wisconsin & SANS Internet Storm Center, USA. DSHIELD logs. 2009, https://www.impactcybertrust.org/dataset_view?idDataset=261.

[89] Hostiadi DP, Ahmad T. Hybrid model for bot group activity detection using similarity and correlation approaches based on network traffic flows analysis. J King Saud Univ Comput Inf Sci 2022;34(7):4219–32. http://dx.doi.org/10.1016/j.jksuci.2022.05.004.

[90] Zhou X, Dillon T. A statistical-heuristic feature selection criterion for decision tree induction. IEEE Trans Pattern Anal Mach Intell 1991;13(8):834–41. http://dx.doi.org/10.1109/34.85676.

[91] Xiong C, Zhu T, Dong W, Ruan L, Yang R, Cheng Y, et al. Conan: A practical real-time APT detection system with high accuracy and efficiency. IEEE Trans Dependable Secur Comput 2022;19(1):551–65. http://dx.doi.org/10.1109/TDSC.2020.2971484.

[92] Deissenboeck F, Juergens E, Lochmann K, Wagner S. Software quality models: Purposes, usage scenarios and requirements. In: 2009 ICSE workshop on software quality. 2009, p. 9–14. http://dx.doi.org/10.1109/WOSQ.2009.5071551.

[93] Scutari M, Panero F, Proissl M. Achieving fairness with a simple ridge penalty. Stat Comput 2022;32(5):77. http://dx.doi.org/10.1007/s11222-022-10143-w.

[94] Ghori KM, Imran M, Nawaz A, Abbasi RA, Ullah A, Szathmary L. Performance analysis of machine learning classifiers for non-technical loss detection. J Ambient Intell Humaniz Comput 2023;14:15327–42. http://dx.doi.org/10.1007/s12652-019-01649-9.

[95] Graham JW. Missing data: Analysis and design. New York: Springer Science & Business Media; 2012, http://dx.doi.org/10.1007/978-1-4614-4018-5.

[96] Pawlicki M, Choraś M, Kozik R, Hoł ubowicz W. Missing and incomplete data handling in cybersecurity applications. In: Intelligent information and database systems: 13th Asian conference, ACIIDS 2021, Phuket, Thailand, April 7–10, 2021, proceedings. Berlin, Heidelberg: Springer-Verlag; 2021, p. 413–26. http://dx.doi.org/10.1007/978-3-030-73280-6_33.

[97] Rainio O, Teuho J, Klén R. Evaluation metrics and statistical tests for machine learning. Sci Rep 2024;14(1):6086. http://dx.doi.org/10.1038/s41598-024-56706-x.

[98] Zeng Z, Peng W, Zhao B. Improving the accuracy of network intrusion detection with causal machine learning. Secur Commun Netw 2021;2021(1):8986243. http://dx.doi.org/10.1155/2021/8986243.

[99] Baba K, Shibata R, Sibuya M. Partial correlation and conditional correlation as measures of conditional independence. Aust N Z J Stat 46(4):657–64. http://dx.doi.org/10.1111/j.1467-842X.2004.00360.x.

[100] Durbin J, Watson GS. Testing for serial correlation in least squares regression. II. Biometrika 1951;38(1):159–77. http://dx.doi.org/10.2307/2332325.

[101] Kuo Y-H, Li Z, Kifer D. Detecting outliers in data with correlated measures. In: Proceedings of the 27th ACM international conference on information and knowledge management. New York, NY, USA: Association for Computing Machinery; 2018, p. 287–96. http://dx.doi.org/10.1145/3269206.3271798.

[102] Vujović Ž. Classification model evaluation metrics. Int J Adv Comput Sci Appl 2021;12(6):599–606. http://dx.doi.org/10.14569/IJACSA.2021.0120670.